

# Predicting Cache Evolution: The Growing Capacity and Latency Cost of CPU Caching

Abbott Sherwin, Ian Ongwacho

August 2026

## 1 Introduction

Modern CPUs include many microarchitecture optimizations that are normally opaque to the end user. However, those optimizations can be revealed using analysis of physical information, such as timing or power usage. In this paper, we present a 3-phase reverse engineering of a range of CPU cache architectures. In Phase 1, we will use timing analysis to determine the size and structure of the L1, L2, and LLC for 8 different systems. In Phase 2, we will verify our results using program counters and the available literature on the processors. Finally, in Phase 3 we will rerun our analysis on NC State's High Performance Computing cluster, to compare consumer-grade hardware to enterprise systems.

| Host      | CPU Model from course slides | ISA/Architecture | Microarchitecture | Year |
|-----------|------------------------------|------------------|-------------------|------|
| Sunbird   | 2× Xeon E5-2680 v3           | x86-64           | Haswell           | 2014 |
| Elevation | 12th Gen i5-1235U            | x86-64           | Alder Lake        | 2021 |
| Skylark   | 2× AMD EPYC 7532             | x86-64           | Zen 2 (Rome)      | 2019 |
| Artemisia | 2× Xeon Gold 5420+           | x86-64           | Raptor Cove       | 2023 |
| Charnwood | Core i7-6700                 | x86-64           | Skylake           | 2015 |
| Crux      | Core i7-9700                 | x86-64           | Coffee Lake       | 2019 |
| Ookay     | Core i7-7700                 | x86-64           | Kaby Lake         | 2017 |
| Upgrade   | Core i7-8700                 | x86-64           | Coffee Lake       | 2017 |

## 2 Phase I

In this phase, we implemented a timing analysis to determine the cache architecture for 8 different consumer-grade processors.

### 2.1 Cache Latency vs. Working Set Size

For our first benchmark, we instantiated a sequential series of addresses in memory, then randomized their order. Beginning at the first node, we accessed the data stored at each location to give us the next location, also known as "pointer chasing". We did this in order to prevent the prefetcher from learning our access pattern and preloading the data. We swept the size of the address set from a few kilobytes up to many megabytes. As the size of the working set increases, we expect to see steps in access time, where the transition occurs when the set is too large to fit in the closest cache to the CPU. In our data, we found that there were 3 plateaus before memory, indicating the presence of a 3 level cache structure, with L1, L2, and LLC. In most cases, the L1 cache was less than 128 KiB, the L2 cache was less than 1 MiB, and the LLC was less than 16 MiB.

```
1 static void make_random_cycle(struct node *nodes, size_t n, uint32_t seed) {
2     if (n < 2) {
3         fprintf(stderr, "Need at least two nodes.\n");
4         exit(EXIT_FAILURE);
5     }
6
7     size_t *order = malloc(n * sizeof(*order));
8     if (!order) {
9         perror("malloc order");
10        exit(EXIT_FAILURE);
11    }
12
13    for (size_t i = 0; i < n; i++) order[i] = i;
14
15    uint32_t state = seed ? seed : 1u;
16    for (size_t i = n - 1; i > 0; i--) {
17        size_t j = (size_t)(xorshift32(&state) % (uint32_t)(i + 1));
18        size_t temp = order[i];
19        order[i] = order[j];
20        order[j] = temp;
21    }
22
23    for (size_t i = 0; i < n; i++) {
24        size_t current = order[i];
25        size_t next = order[(i + 1) % n];
26        nodes[current].next = &nodes[next];
27    }
28
29    free(order);
30 }
```

Listing 1: Code to create cycle of memory locations

```

1 static double chase_batch(struct node **cursor, uint64_t steps) {
2     struct node *p = *cursor;
3
4     compiler_barrier();
5     uint64_t t0 = x86_tsc_start();
6
7     for (uint64_t i = 0; i < steps; i++) {
8         p = p->next;
9     }
10
11     uint64_t t1 = x86_tsc_stop();
12     compiler_barrier();
13
14     sink_node = p;
15     *cursor = p;
16     return (double)(t1 - t0) / (double)steps;
17 }

```

Listing 2: Code to chase pointers and collect times

```

1 static stats_t run_one_mode(struct node *nodes, size_t node_count,
2                             size_t requested_bytes, size_t actual_bytes,
3                             uint64_t sample_count, uint64_t steps,
4                             const char *mode, uint32_t seed,
5                             const char *prefix, FILE *summary) {
6     if (strcmp(mode, "random") == 0) {
7         make_random_cycle(nodes, node_count, seed);
8     } else if (strcmp(mode, "sequential") == 0) {
9         make_sequential_cycle(nodes, node_count);
10    } else {
11        fprintf(stderr, "Internal error: invalid mode %s\n", mode);
12        exit(EXIT_FAILURE);
13    }
14
15    struct node *cursor = &nodes[0];
16
17    /* Four complete untimed warm-up passes. */
18    uint64_t warm_steps = node_count > UINT64_MAX / 4ULL
19                        ? UINT64_MAX
20                        : (uint64_t)node_count * 4ULL;
21    warm_chain(&cursor, warm_steps);
22
23    if (sample_count > SIZE_MAX / sizeof(double)) {
24        fprintf(stderr, "Sample buffer is too large.\n");
25        exit(EXIT_FAILURE);
26    }
27
28    double *samples = malloc((size_t)sample_count * sizeof(*samples));
29    if (!samples) {
30        fprintf(stderr, "Could not allocate sample buffer for %" PRIu64 " samples.\n",
31                sample_count);
32        exit(EXIT_FAILURE);
33    }
34
35    /* Critical timing loop: intentionally no file I/O here. */
36    for (uint64_t sample = 0; sample < sample_count; sample++) {
37        samples[sample] = chase_batch(&cursor, steps);
38    }
39
40    stats_t statistics = compute_stats(samples, (size_t)sample_count);
41
42    char raw_path[1024];
43    int written = snprintf(raw_path, sizeof(raw_path),
44                          "%s_%zu_%s_raw.csv", prefix, actual_bytes, mode);
45
46    if (written < 0 || (size_t)written >= sizeof(raw_path)) {
47        fprintf(stderr, "Output path is too long.\n");
48        free(samples);
49        exit(EXIT_FAILURE);
50    }
51
52    write_raw_csv(raw_path, samples, sample_count);
53
54    fprintf(summary,
55            "%s,%zu,%zu,%zu,"
56            "%" PRIu64 " ,%" PRIu64 " ,%" PRIu64 " ,%"
57            "%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,"
58            "%" PRIu64 " ,%" PRIu32 " ,%s\n",
59            mode, requested_bytes, actual_bytes, node_count,
60            sample_count, steps, sample_count * steps,
61            statistics.mean, statistics.median, statistics.stddev,
62            statistics.q1, statistics.q3, statistics.p5, statistics.p95,
63            statistics.min, statistics.max, statistics.outliers,
64            seed, raw_path);
65

```

```

66     fflush(summary);
67
68     fprintf(stderr,
69             "bytes=%zu mode=%s mean=%.4f median=%.4f "
70             "p5=%.4f p95=%.4f raw=%s\n",
71             actual_bytes, mode, statistics.mean, statistics.median,
72             statistics.p5, statistics.p95, raw_path);
73
74     free(samples);
75     return statistics;
76 }

```

Listing 3: Code to run one cache size sample

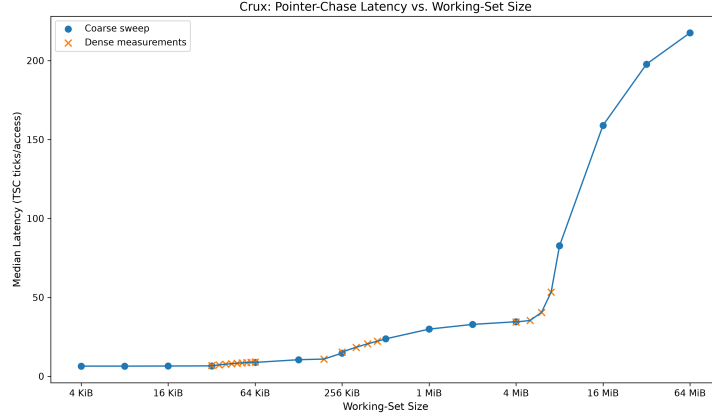


Figure 1: Crux cache latency

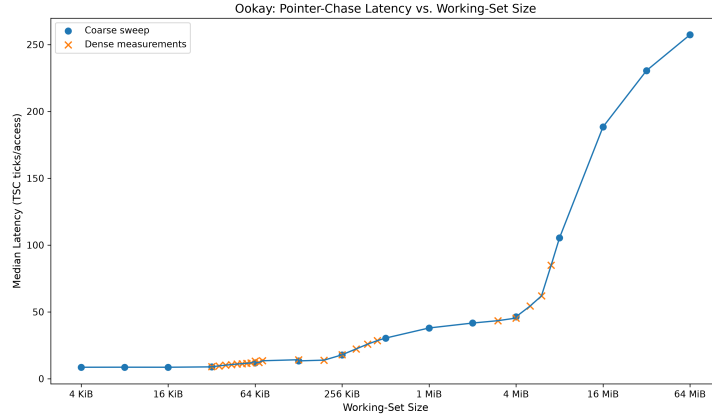


Figure 2: Ookay cache latency

## 2.2 Transitional Behavior

In order to examine more closely the size of each cache level, we ran a dense sweep over the predicted boundaries between levels. This sweep allowed us to see the latency in more detail, and determine with more accuracy the size of each cache level. These dense sweeps are marked on the plots in Section 2.1 with orange crosses. We swept in a range of 0.5-2x the predicted transition, in steps of around 1/8 of the range. As the size of the working set increases past the size of the cache level, a larger and larger fraction of memory accesses cause cache misses, meaning data must be evicted from the cache and loaded from the next level, causing slower accesses.

## 2.3 Randomized vs. Sequential Latency

In phase I we compared randomized dependent pointer-chasing with sequential memory traversal at representative working-set sizes. The randomized access pattern reduces the effectiveness of hardware prefetching because each subsequent memory address depends on the result of the previous load. On the other hand, sequential traversal presents a predictable access pattern that can benefit from spatial locality and hardware prefetching. Median TSC ticks per access were calculated across the two access patterns. The resulting latency differences were used to evaluate the effect of access predictability and to support interpretation of cache-capacity transition observed in the random pointer chase measurements.

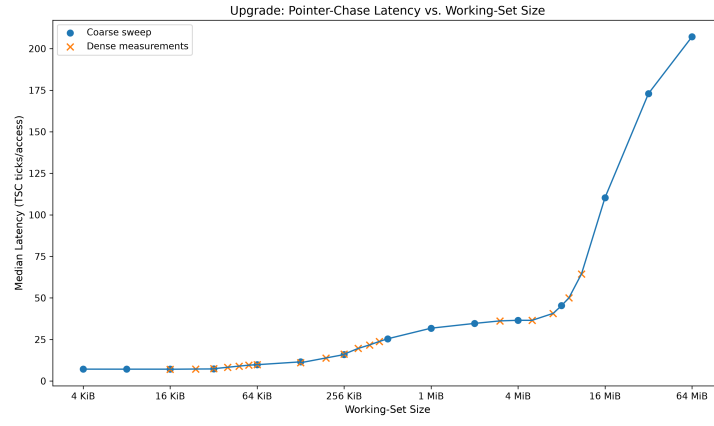


Figure 3: Upgrade cache latency

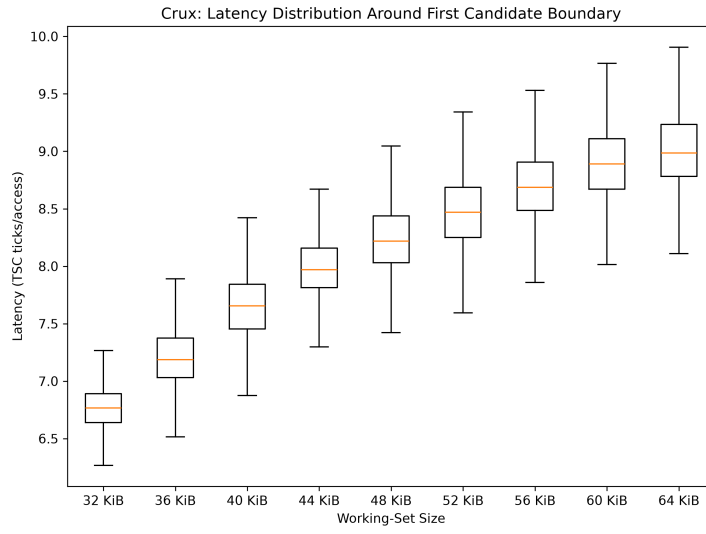


Figure 4: Crux L1-L2 transition

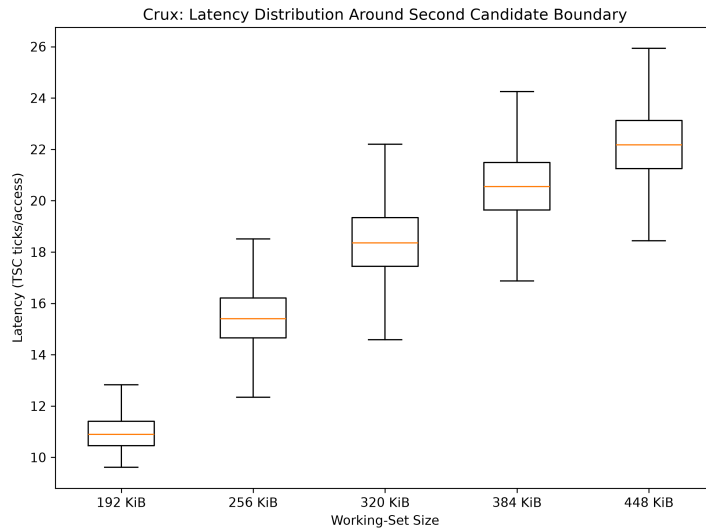


Figure 5: Crux L2-LLC transition

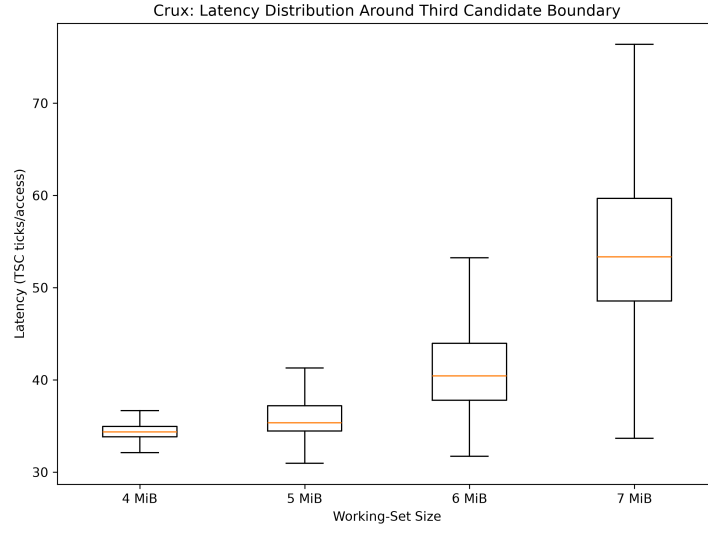


Figure 6: Crux LLC-memory transition

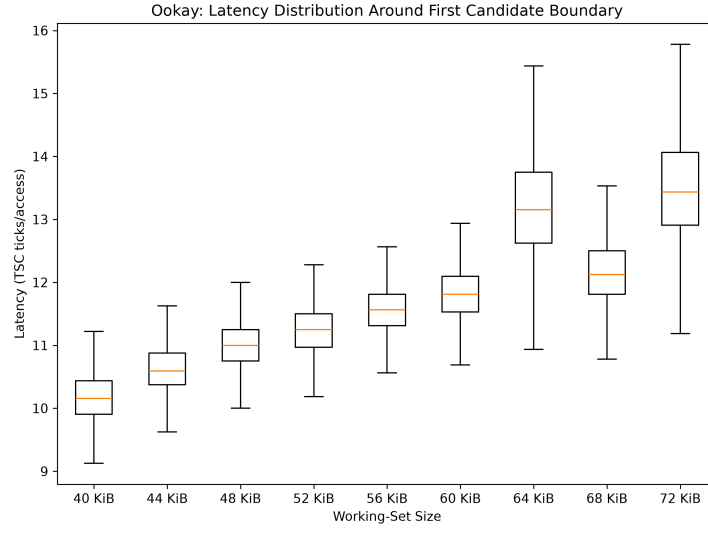


Figure 7: Ookay L1-L2 transition

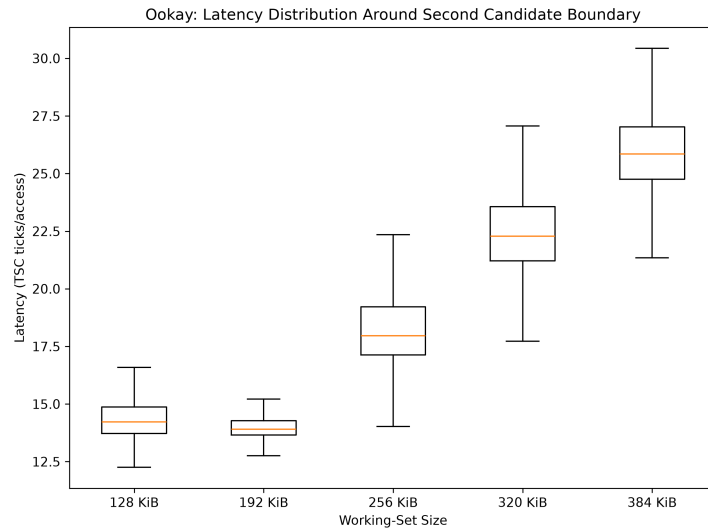


Figure 8: Ookay L2-LLC transition

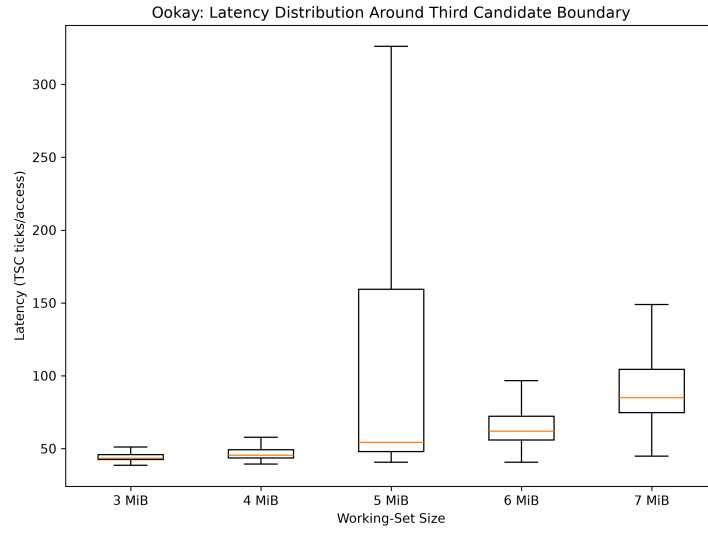


Figure 9: Ookay LLC-memory transition

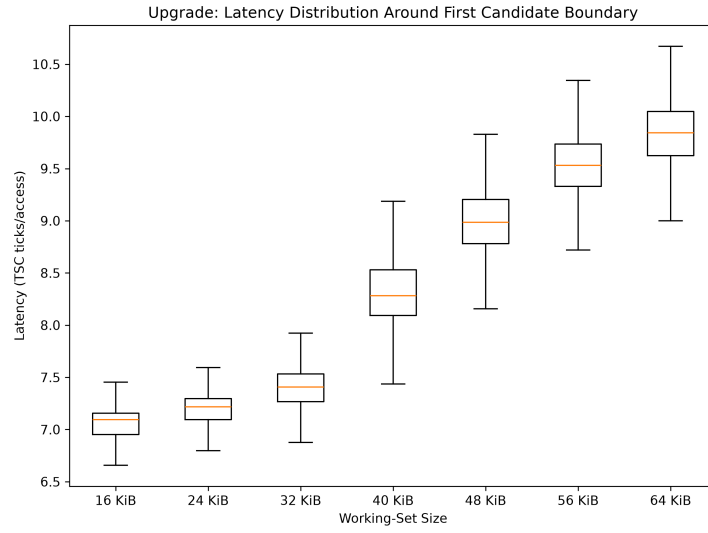


Figure 10: Upgrade L1-L2 transition

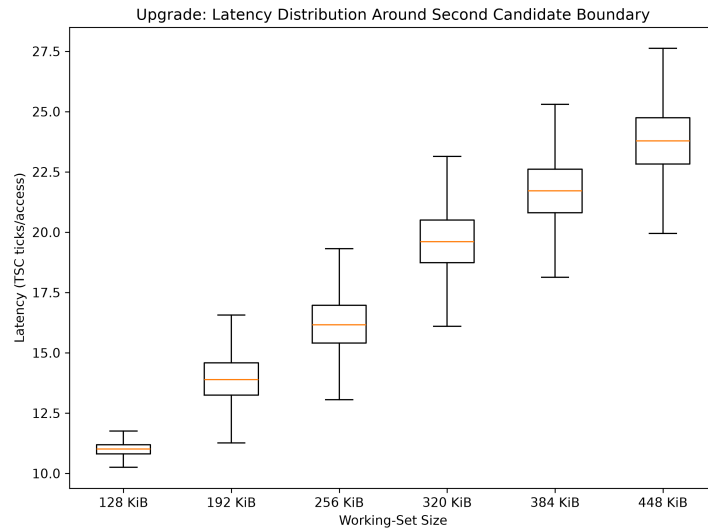


Figure 11: Upgrade L2-LLC transition

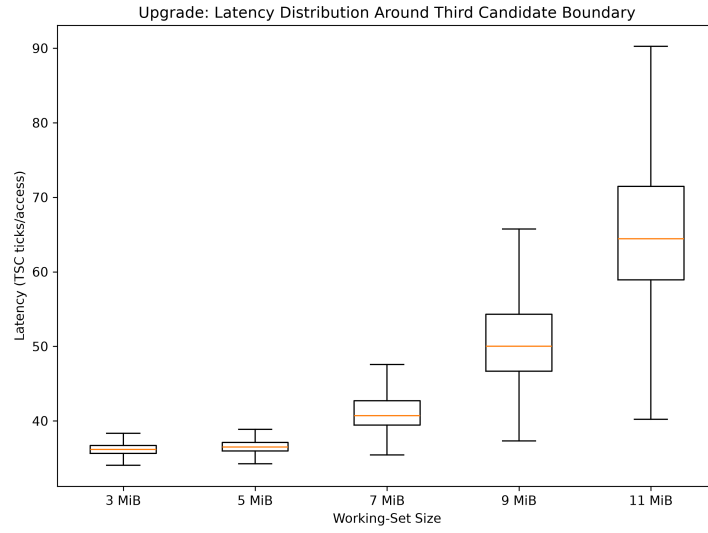


Figure 12: Upgrade LLC-memory transition

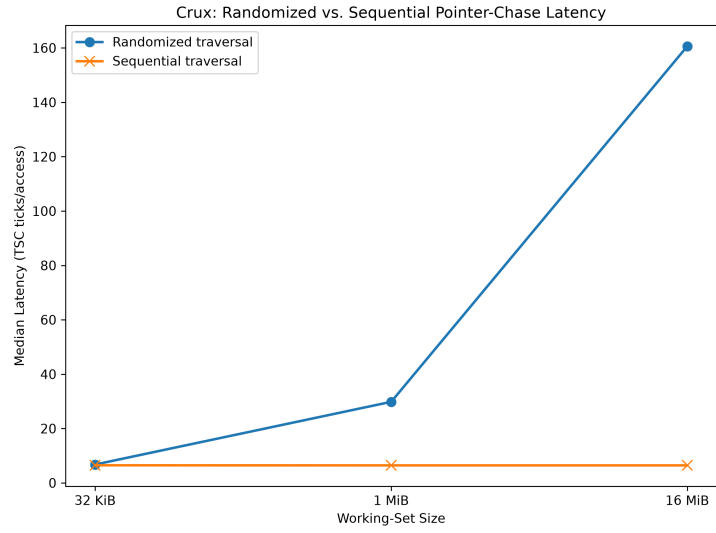


Figure 13: Crux randomized vs sequential latency

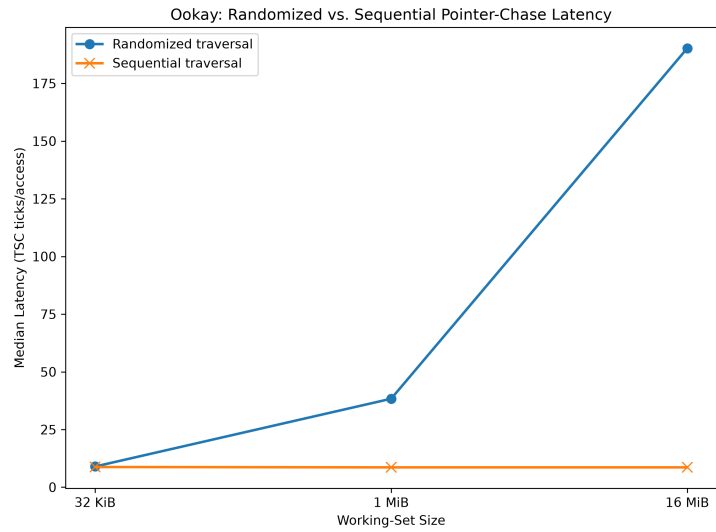


Figure 14: Ookay randomized vs sequential latency

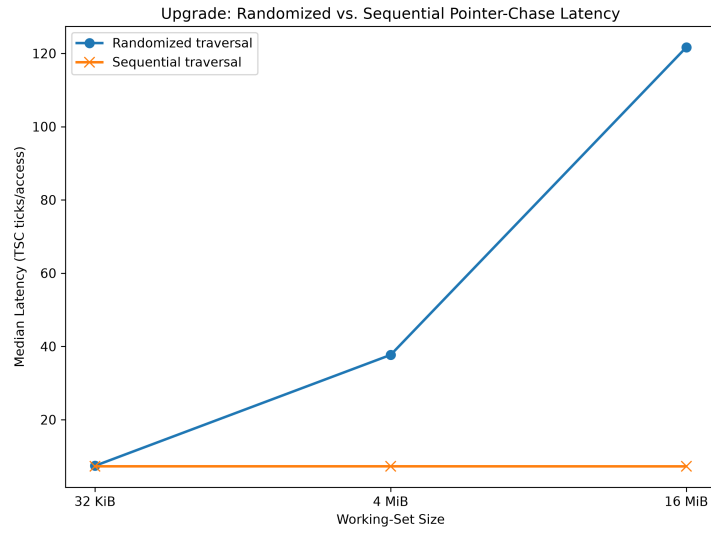


Figure 15: Upgrade randomized vs sequential latency

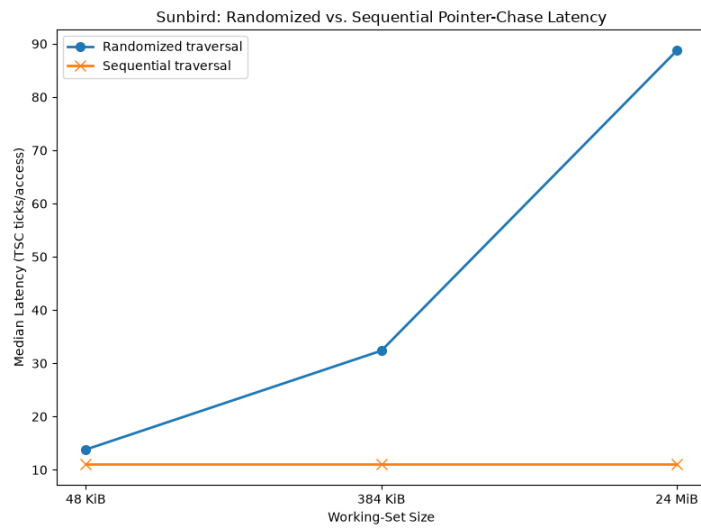


Figure 16: Sunbird randomized vs sequential latency

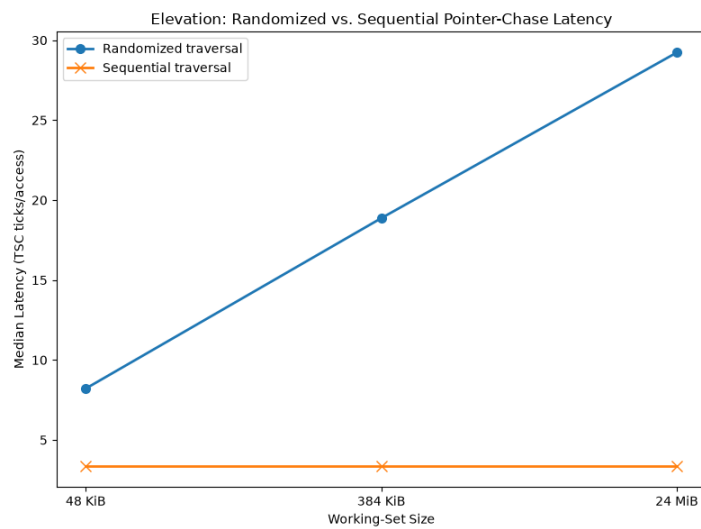


Figure 17: Elevation randomized vs sequential latency

## 2.4 Cache Line Size

To test the size of a cache line, we ran a set of sweeps of working set sizes around the cache level transition. Each sweep of working set sizes tested memory accesses that differed by a tested stride, such as 8, 16, 32, 64, 128 locations. If we saw that increasing the tested stride made latency worse, then we knew that the memory access was no longer close enough to an address already within the cache to have been fetched in a block. We tested values close to the predicted value to narrow down the exact size of the cache line, and found that every tested processor had a line size of 64 bytes for all cache levels.

```

1 static stats_t run_stride_point(size_t stride, size_t requested_footprint,
2                               uint64_t sample_count, uint64_t steps,
3                               uint32_t seed, const char *prefix, FILE *summary) {
4     size_t node_count = requested_footprint / stride;
5     if (node_count < 2) {
6         fprintf(stderr,
7             "Skipping stride=%zu footprint=%zu: fewer than 2 nodes result.\n",
8             stride, requested_footprint);
9         stats_t empty;
10        memset(&empty, 0, sizeof(empty));
11        return empty;
12    }
13
14    size_t actual_footprint = node_count * stride;
15    unsigned char *buffer = allocate_bytes(actual_footprint);
16
17    /* First-touch allocation after the process has been externally pinned. */
18    memset(buffer, 0, actual_footprint);
19
20    build_strided_random_cycle(buffer, stride, node_count, seed);
21
22    struct node *cursor = NODE_AT(buffer, stride, 0);
23
24    uint64_t warm_steps = node_count > UINT64_MAX / 4ULL
25                        ? UINT64_MAX
26                        : (uint64_t)node_count * 4ULL;
27    warm_chain(&cursor, warm_steps);
28
29    if (sample_count > SIZE_MAX / sizeof(double)) {
30        fprintf(stderr, "Sample buffer is too large.\n");
31        exit(EXIT_FAILURE);
32    }
33
34    double *samples = malloc((size_t)sample_count * sizeof(*samples));
35    if (!samples) {
36        fprintf(stderr, "Could not allocate sample buffer for %" PRIu64 " samples.\n",
37            sample_count);
38        exit(EXIT_FAILURE);
39    }
40
41    /* Critical timing loop: intentionally no file I/O here. */
42    for (uint64_t sample = 0; sample < sample_count; sample++) {
43        samples[sample] = chase_batch(&cursor, steps);
44    }
45
46    stats_t statistics = compute_stats(samples, (size_t)sample_count);
47
48    char raw_path[1024];
49    int written = snprintf(raw_path, sizeof(raw_path),
50                          "%s_stride%zu_footprint%zu_raw.csv",
51                          prefix, stride, actual_footprint);
52    if (written < 0 || (size_t)written >= sizeof(raw_path)) {
53        fprintf(stderr, "Output path is too long.\n");
54        free(samples);
55        free_bytes(buffer);
56        exit(EXIT_FAILURE);
57    }
58
59    write_raw_csv(raw_path, samples, sample_count);
60
61    fprintf(summary,
62        "%zu,%zu,%zu,%zu,"
63        "%" PRIu64 ",%" PRIu64 ",%" PRIu64 ",%"
64        "%9f,%9f,%9f,%9f,%9f,%9f,%9f,%9f,%9f,%9f,"
65        "%" PRIu64 ",%" PRIu32 ",%s\n",
66        stride, requested_footprint, actual_footprint, node_count,
67        sample_count, steps, sample_count * steps,
68        statistics.mean, statistics.median, statistics.stddev,
69        statistics.q1, statistics.q3, statistics.p5, statistics.p95,
70        statistics.min, statistics.max, statistics.outliers,
71        seed, raw_path);
72    fflush(summary);
73
74    fprintf(stderr,
75        "stride=%zu footprint=%zu nodes=%zu mean=%.4f median=%.4f "
76        "p5=%.4f p95=%.4f raw=%s\n",
77        stride, actual_footprint, node_count,
78        statistics.mean, statistics.median,
79        statistics.p5, statistics.p95, raw_path);
80
81    free(samples);

```

```

82     free_bytes(buffer);
83     return statistics;
84 }

```

Listing 4: Line size benchmark

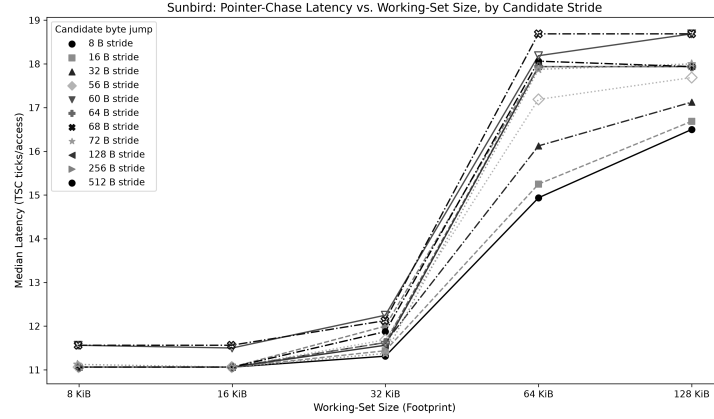


Figure 18: Sunbird L1 line size

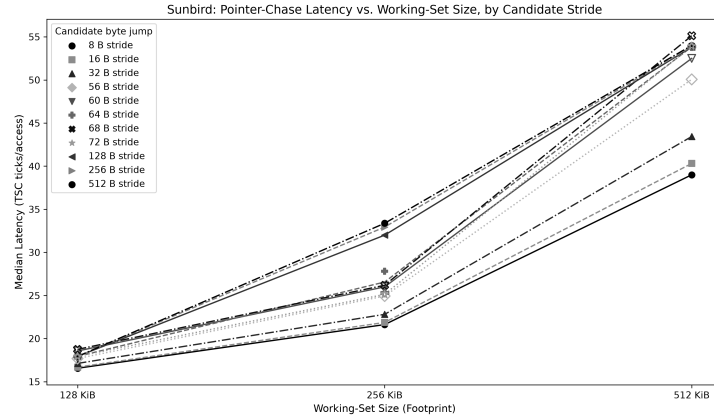


Figure 19: Sunbird L2 line size

## 2.5 Associativity

To determine associativity, we created a small set of congruent addresses. Using the same pointer chasing randomization as earlier, we swept the size of the working set. When we see a sharp increase in latency, this indicates that the cache has run out of ways to store all the data, indicating a conflict miss. The size at which the spike occurs is the associativity of the cache.

```

1 static stats_t run_assoc_point(unsigned char *buffer, size_t set_stride, size_t ways,
2                               uint64_t sample_count, uint64_t steps,
3                               uint32_t seed, const char *prefix, FILE *summary) {
4     if (ways < 1) {
5         fprintf(stderr, "WAYS must be at least 1.\n");
6         exit(EXIT_FAILURE);
7     }
8
9     if (ways == 1) {
10        /* A one-node "cycle" (self-loop) is a degenerate case; use it
11         * only as a same-line-reload baseline and skip pointer-cycle
12         * construction that requires >= 2 nodes. */
13        struct node *only = NODE_AT(buffer, set_stride, 0);
14        memset(only, 0, sizeof(*only));
15        only->next = only;
16
17        struct node *cursor = only;
18        uint64_t warm_steps = 64;
19        warm_chain(&cursor, warm_steps);
20
21        double *samples = malloc((size_t)sample_count * sizeof(*samples));
22        if (!samples) {
23            fprintf(stderr, "Could not allocate sample buffer.\n");
24            exit(EXIT_FAILURE);

```

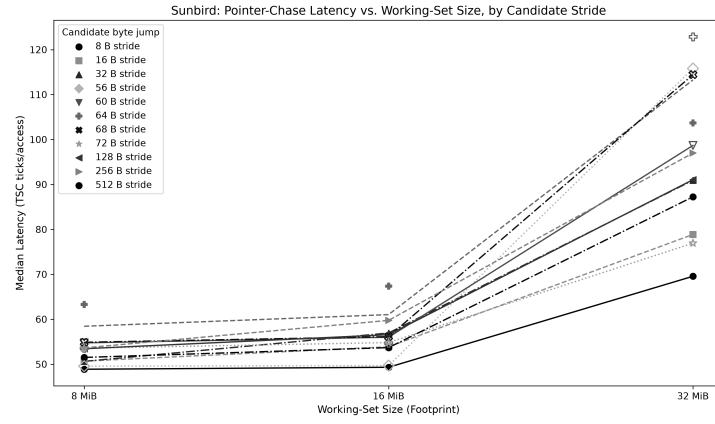


Figure 20: Sunbird LLC line size

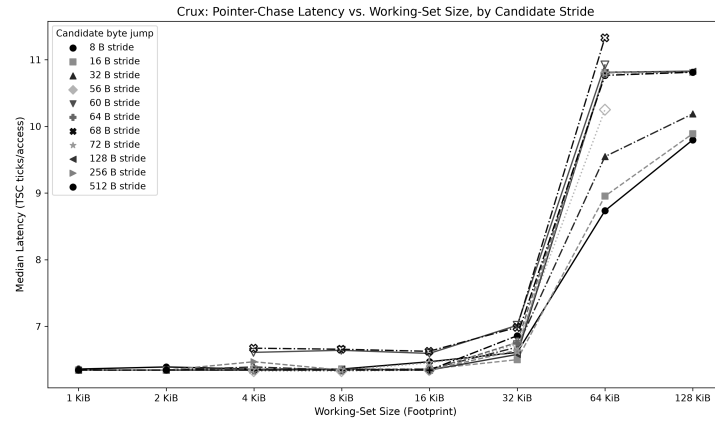


Figure 21: Crux L1 line size

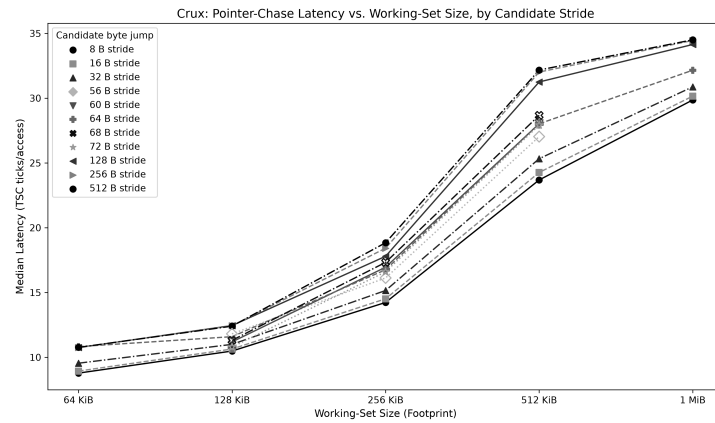


Figure 22: Crux L2 line size

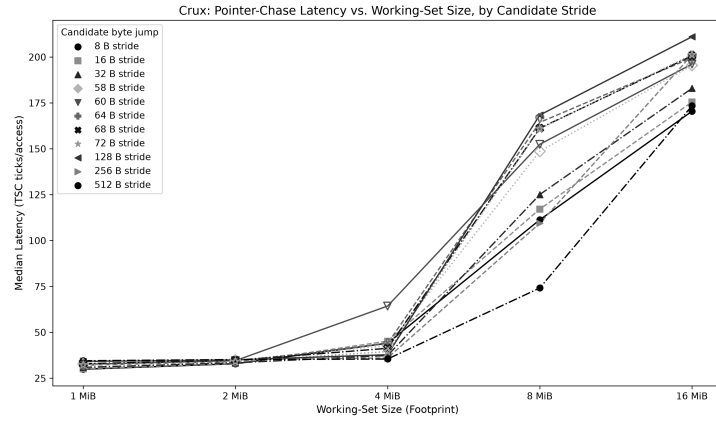


Figure 23: Crux LLC line size

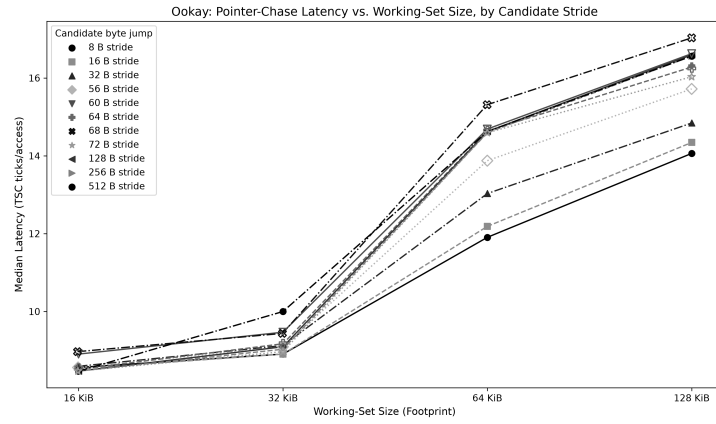


Figure 24: Ookay L1 line size

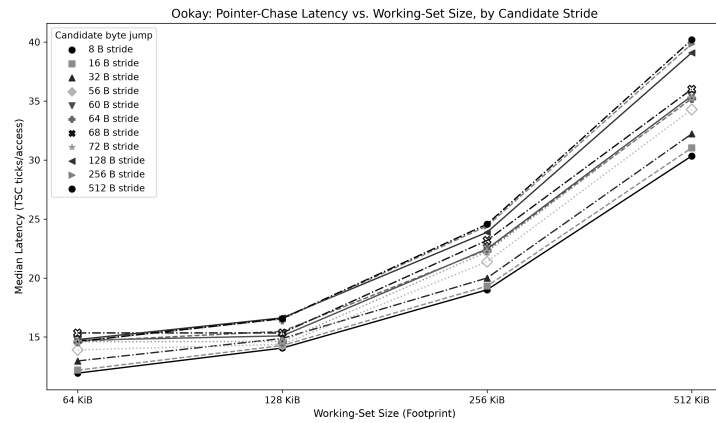


Figure 25: Ookay L2 line size

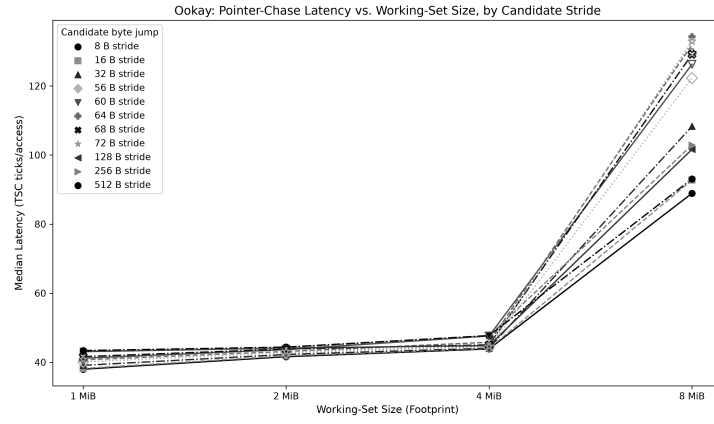


Figure 26: Ookay LLC line size

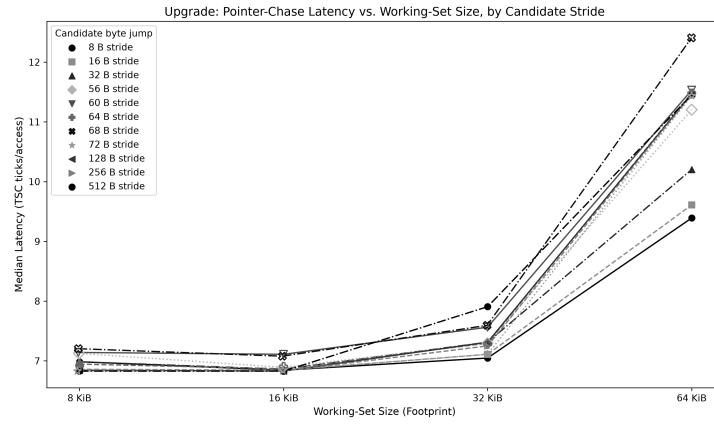


Figure 27: Upgrade L1 line size

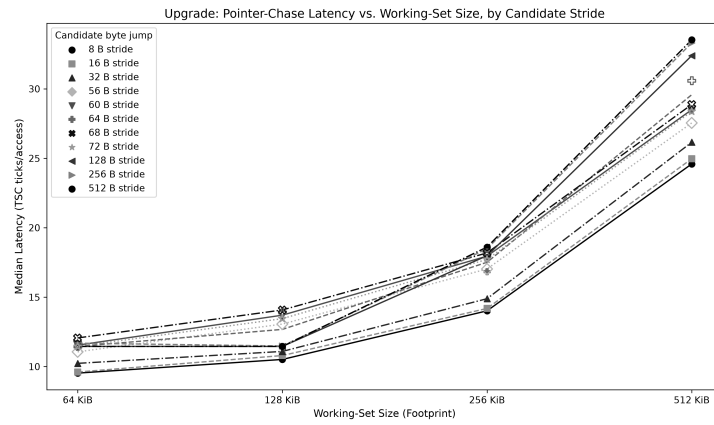


Figure 28: Upgrade L2 line size

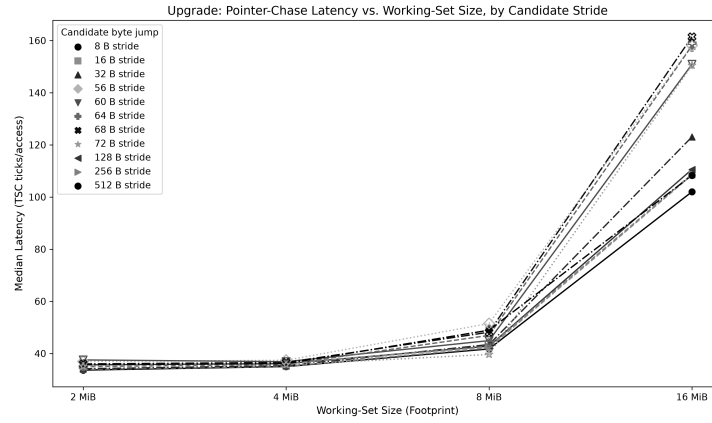


Figure 29: Upgrade LLC line size

```

25     }
26     for (uint64_t sample = 0; sample < sample_count; sample++) {
27         samples[sample] = chase_batch(&cursor, steps);
28     }
29
30     stats_t statistics = compute_stats(samples, (size_t)sample_count);
31
32     char raw_path[1024];
33     int written = snprintf(raw_path, sizeof(raw_path),
34                           "%s_setstride%zu_ways%zu_raw.csv",
35                           prefix, set_stride, ways);
36     if (written < 0 || (size_t)written >= sizeof(raw_path)) {
37         fprintf(stderr, "Output path is too long.\n");
38         free(samples);
39         exit(EXIT_FAILURE);
40     }
41     write_raw_csv(raw_path, samples, sample_count);
42
43     fprintf(summary,
44             "%zu,%zu,"
45             "%u PRIu64 ", "%u PRIu64 ", "%u PRIu64 ", "
46             "%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,"
47             "%u PRIu64 ", "%u PRIu32 ", %s\n",
48             set_stride, ways,
49             sample_count, steps, sample_count * steps,
50             statistics.mean, statistics.median, statistics.stddev,
51             statistics.q1, statistics.q3, statistics.p5, statistics.p95,
52             statistics.min, statistics.max, statistics.outliers,
53             seed, raw_path);
54     fflush(summary);
55
56     fprintf(stderr,
57             "set_stride=%zu ways=%zu mean=%.4f median=%.4f p5=%.4f p95=%.4f raw=%s\n",
58             set_stride, ways, statistics.mean, statistics.median,
59             statistics.p5, statistics.p95, raw_path);
60
61     free(samples);
62     return statistics;
63 }
64
65 /* Re-touch and rebuild the cycle over exactly the first 'ways'
66  * congruent addresses so their absolute addresses (and therefore
67  * the address bits presumed to select the cache set) stay fixed
68  * across every point in the sweep. */
69 memset(buffer, 0, ways * set_stride);
70 build_strided_random_cycle(buffer, set_stride, ways, seed);
71
72 struct node *cursor = NODE_AT(buffer, set_stride, 0);
73
74 /* Warm through several full passes of the conflict cycle so the
75  * measurement reflects steady-state conflict behavior rather than
76  * first-touch cold misses. */
77 uint64_t warm_steps = (uint64_t)ways * 8ULL;
78 warm_chain(&cursor, warm_steps);
79
80 if (steps % ways != 0) {
81     fprintf(stderr,
82             "NOTE: STEPS=%u PRIu64 " is not a multiple of WAYS=%zu; "
83             "each timed batch will not cover a whole number of passes "
84             "through the conflict cycle.\n",
85             steps, ways);
86 }
87
88 if (sample_count > SIZE_MAX / sizeof(double)) {
89     fprintf(stderr, "Sample buffer is too large.\n");

```

```

90     exit(EXIT_FAILURE);
91 }
92
93 double *samples = malloc((size_t)sample_count * sizeof(*samples));
94 if (!samples) {
95     fprintf(stderr, "Could not allocate sample buffer for %" PRIu64 " samples.\n",
96         sample_count);
97     exit(EXIT_FAILURE);
98 }
99
100 /* Critical timing loop: intentionally no file I/O here. */
101 for (uint64_t sample = 0; sample < sample_count; sample++) {
102     samples[sample] = chase_batch(&cursor, steps);
103 }
104
105 stats_t statistics = compute_stats(samples, (size_t)sample_count);
106
107 char raw_path[1024];
108 int written = snprintf(raw_path, sizeof(raw_path),
109     "%s_setstride%zu_ways%zu_raw.csv",
110     prefix, set_stride, ways);
111 if (written < 0 || (size_t)written >= sizeof(raw_path)) {
112     fprintf(stderr, "Output path is too long.\n");
113     free(samples);
114     exit(EXIT_FAILURE);
115 }
116
117 write_raw_csv(raw_path, samples, sample_count);
118
119 fprintf(summary,
120     "%zu,%zu,"
121     "%" PRIu64 " ,%" PRIu64 " ,%" PRIu64 " ,%"
122     "%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,%.9f,"
123     "%" PRIu64 " ,%" PRIu32 " ,%s\n",
124     set_stride, ways,
125     sample_count, steps, sample_count * steps,
126     statistics.mean, statistics.median, statistics.stddev,
127     statistics.q1, statistics.q3, statistics.p5, statistics.p95,
128     statistics.min, statistics.max, statistics.outliers,
129     seed, raw_path);
130 fflush(summary);
131
132 fprintf(stderr,
133     "set_stride=%zu ways=%zu mean=%.4f median=%.4f p5=%.4f p95=%.4f raw=%s\n",
134     set_stride, ways, statistics.mean, statistics.median,
135     statistics.p5, statistics.p95, raw_path);
136
137 free(samples);
138 return statistics;
139 }

```

Listing 5: Associativity benchmark

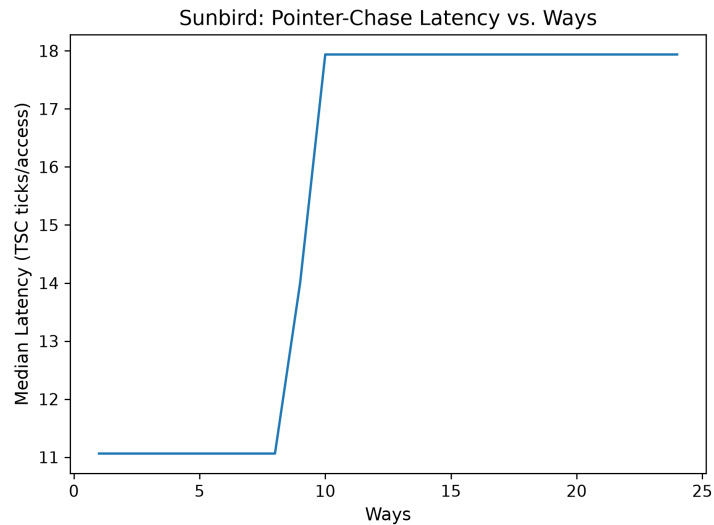


Figure 30: Sunbird L1 associativity

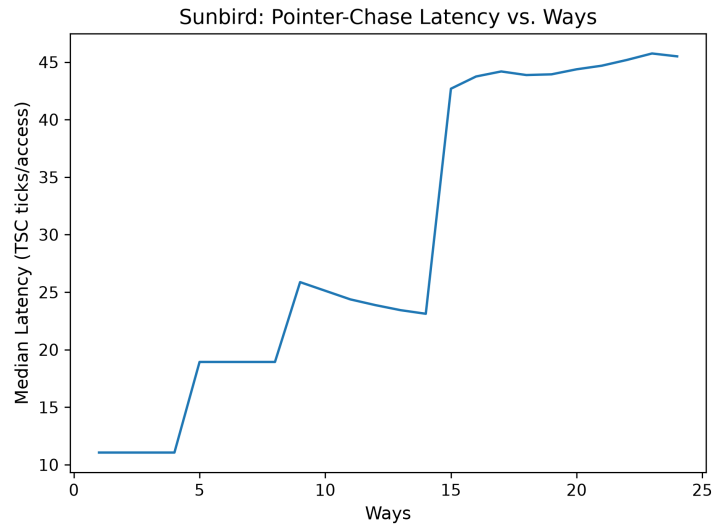


Figure 31: Sunbird L2 associativity

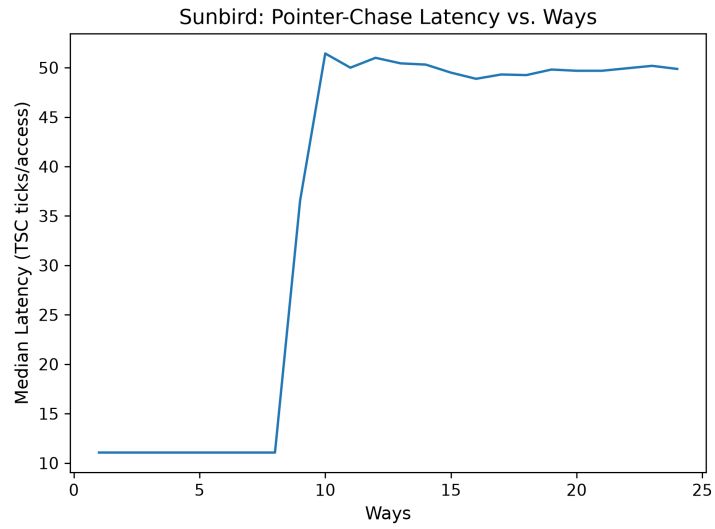


Figure 32: Sunbird LLC associativity

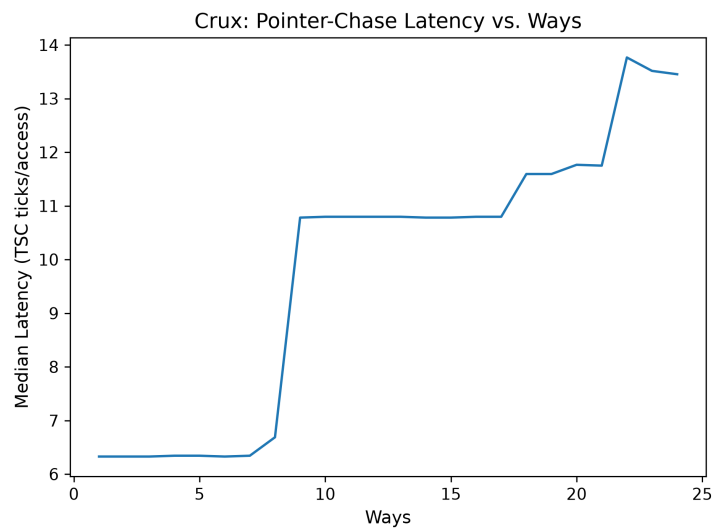


Figure 33: Crux L1 associativity

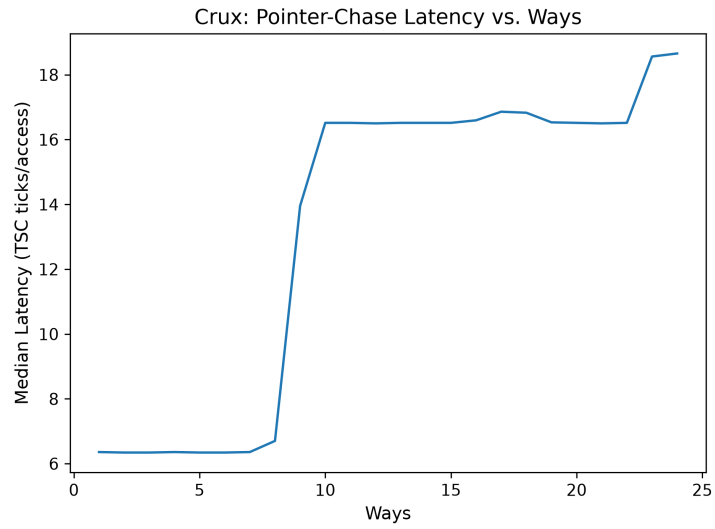


Figure 34: Crux L2 associativity

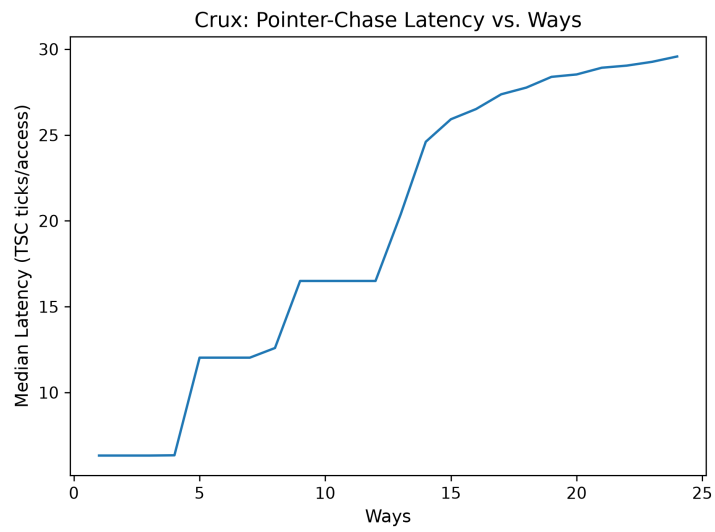


Figure 35: Crux LLC associativity

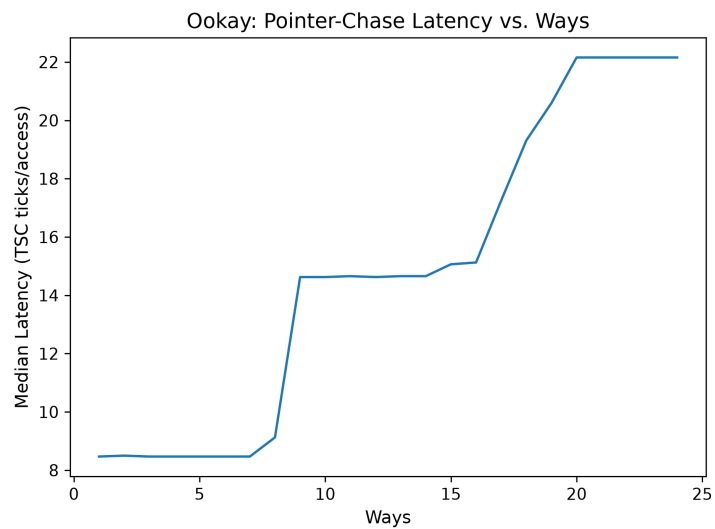


Figure 36: Ookay L1 associativity

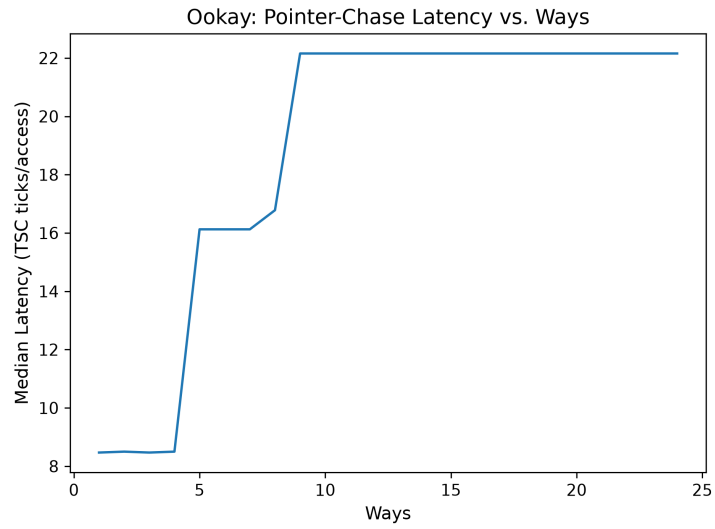


Figure 37: Ookay L2 associativity

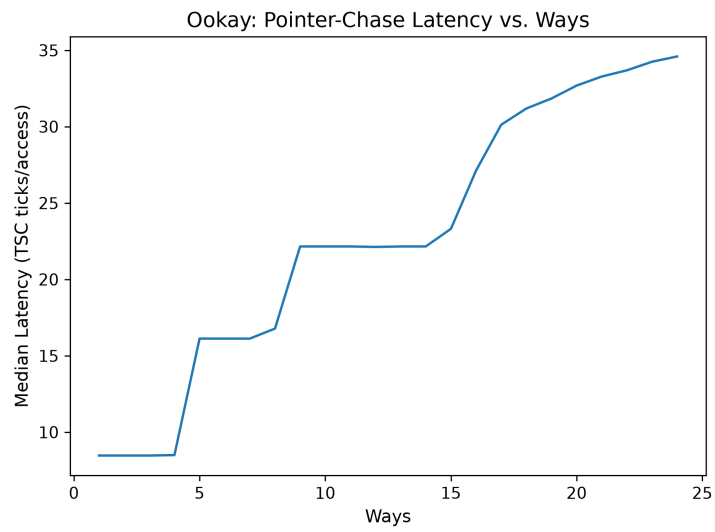


Figure 38: Ookay LLC associativity

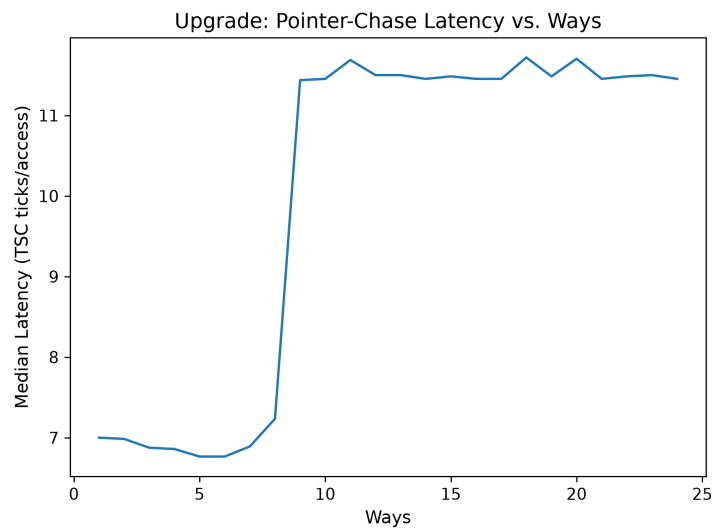


Figure 39: Upgrade L1 associativity

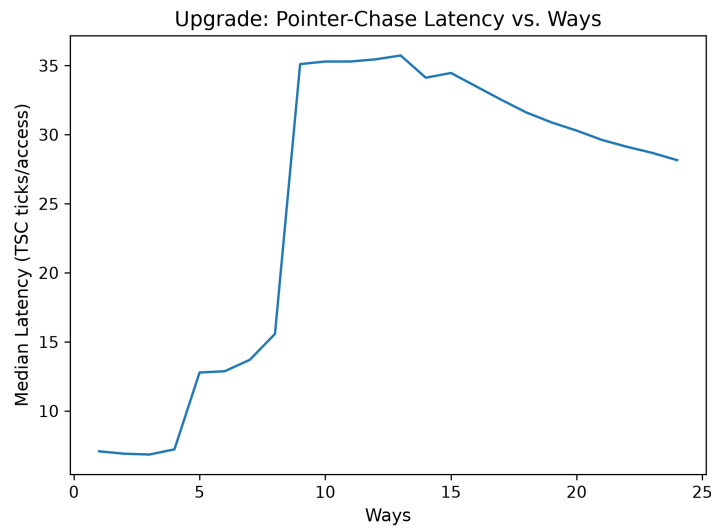


Figure 40: Upgrade L2 associativity

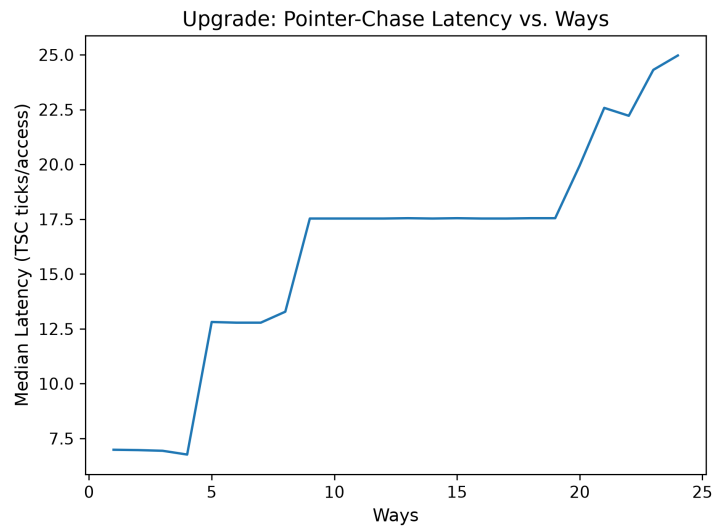


Figure 41: Upgrade LLC associativity

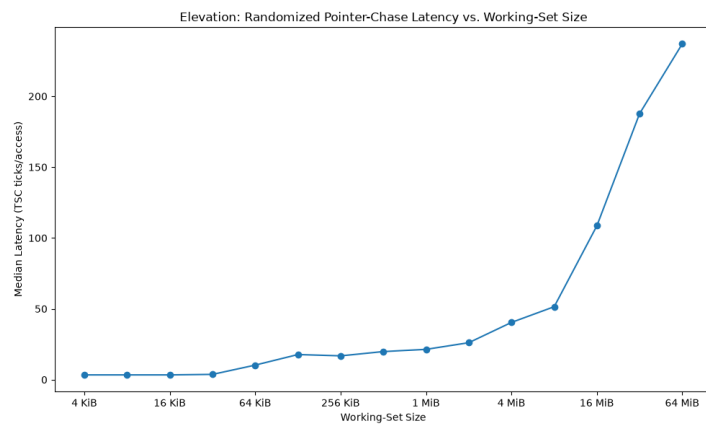


Figure 42: Elevation coarse capacity sweep.

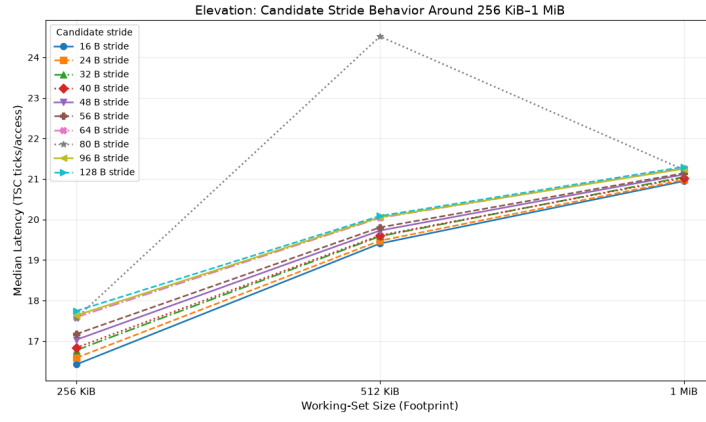


Figure 43: Elevation coarse and dense working-set measurements.

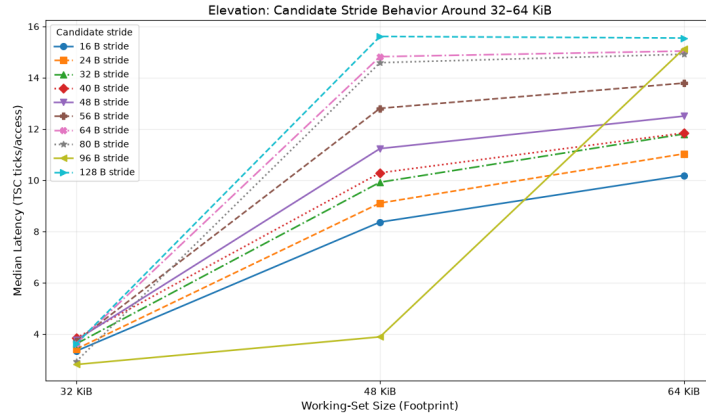


Figure 44: Elevation dense measurements around the first candidate boundary.

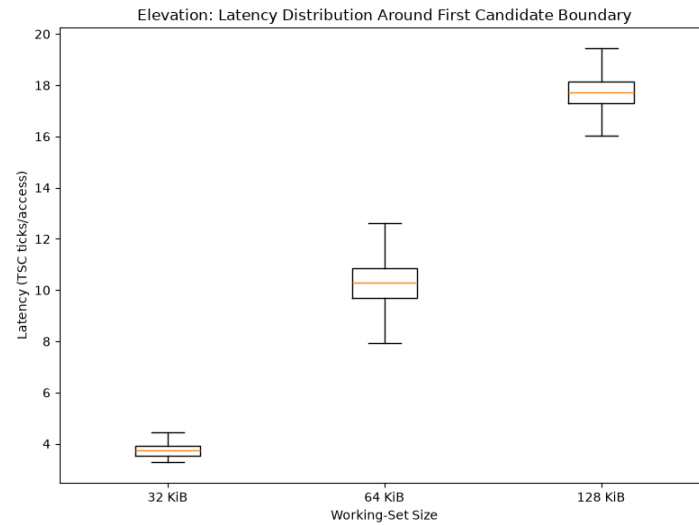


Figure 45: Elevation latency distribution around the first candidate boundary.

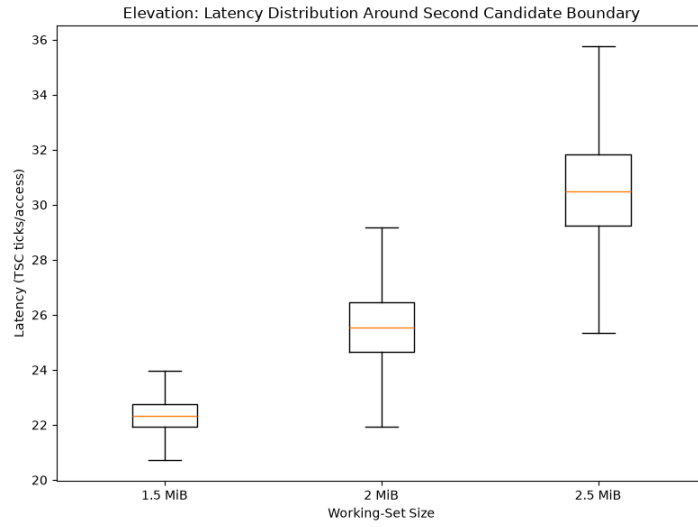


Figure 46: Elevation latency distribution around the second candidate boundary.

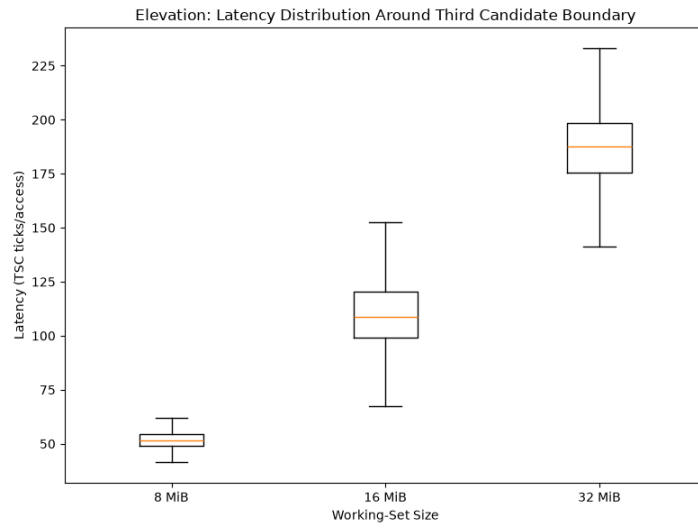


Figure 47: Elevation latency distribution around the third candidate boundary.

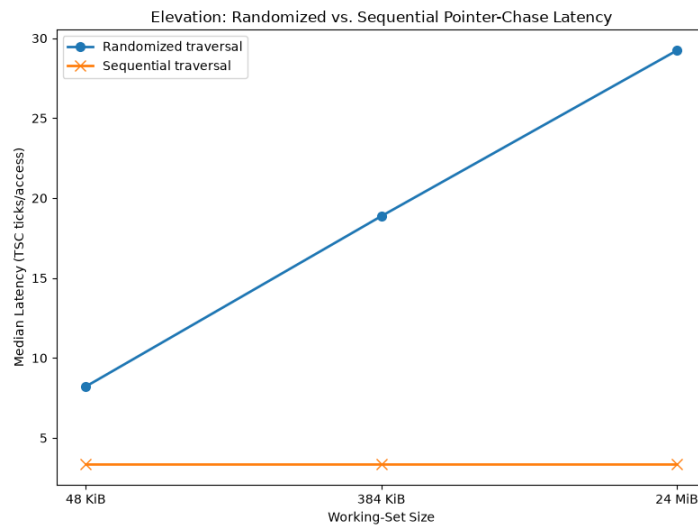


Figure 48: Elevation randomized versus sequential pointer-chase latency.

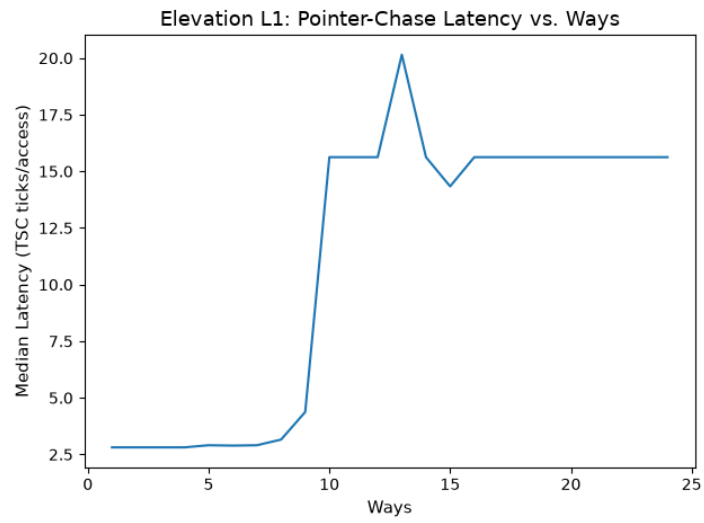


Figure 49: Elevation L1 pointer-chase latency versus number of ways.

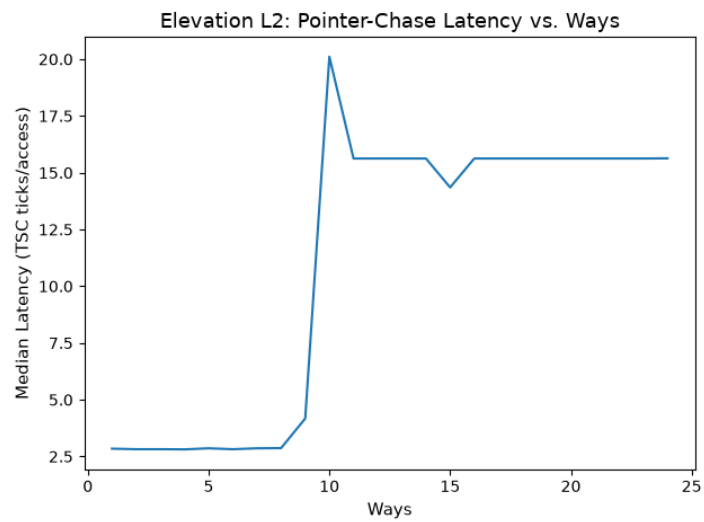


Figure 50: Elevation L2 pointer-chase latency versus number of ways.

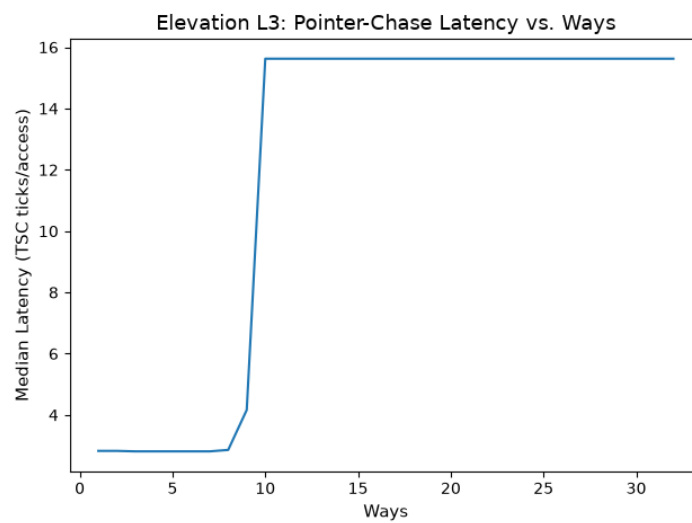


Figure 51: Elevation LLC pointer-chase latency versus number of ways.

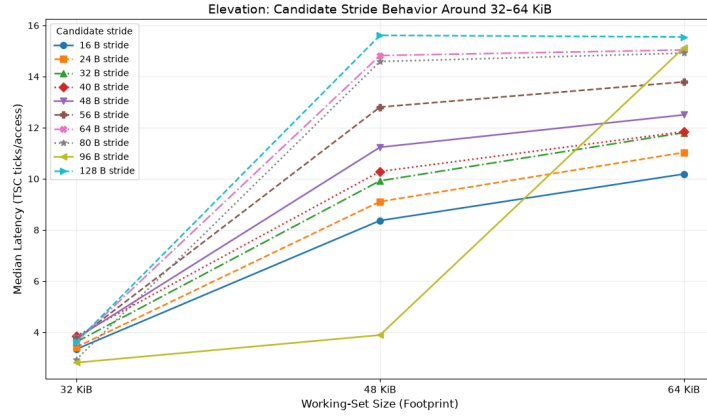


Figure 52: Elevation candidate stride behavior around the 32-64 KiB working-set range.

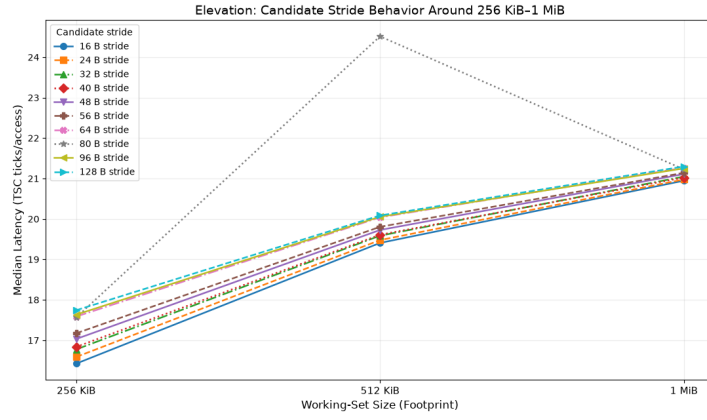


Figure 53: Elevation candidate stride behavior around the 256 KiB-1 MiB working-set range.

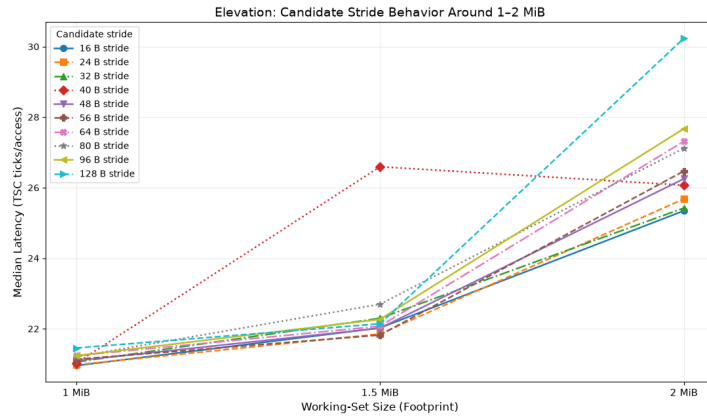


Figure 54: Elevation candidate stride behavior around the 1-2 MiB working-set range.

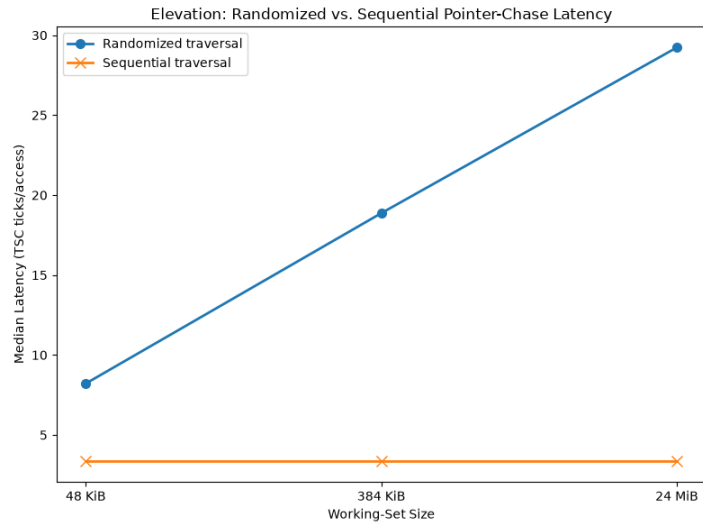


Figure 55: Elevation randomized versus sequential pointer-chase latency.

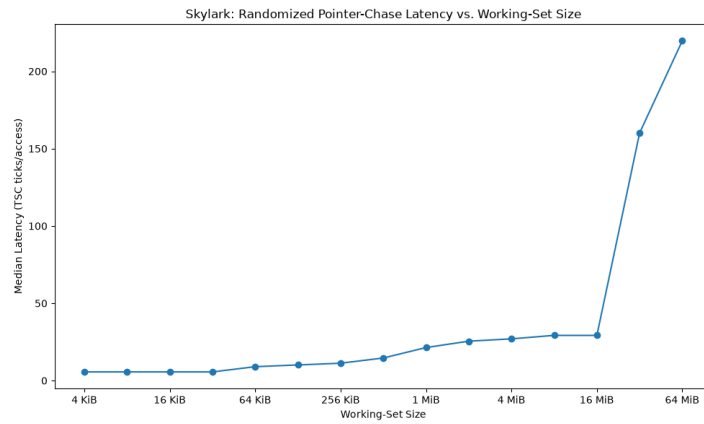


Figure 56: Skylark coarse capacity sweep.

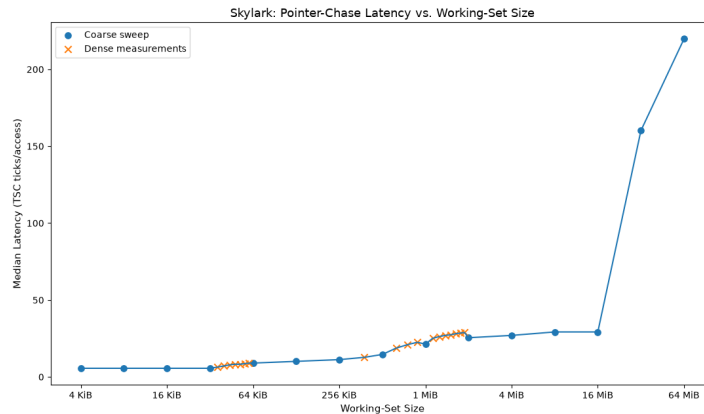


Figure 57: Skylark coarse and dense working-set measurements.

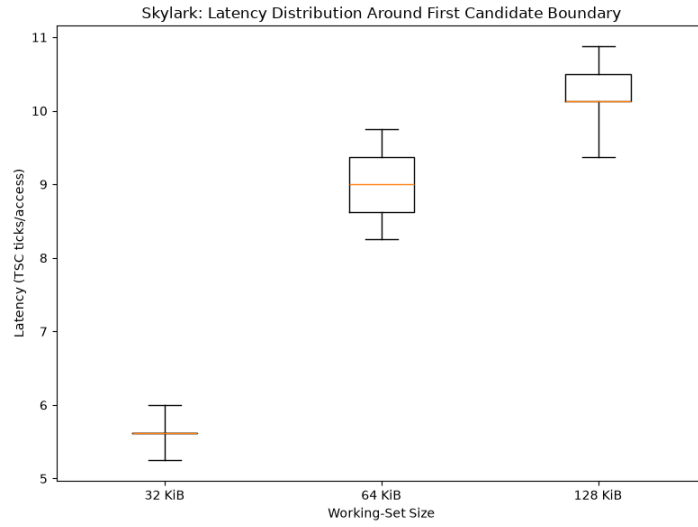


Figure 58: Skylark latency distribution around the first candidate cache boundary.

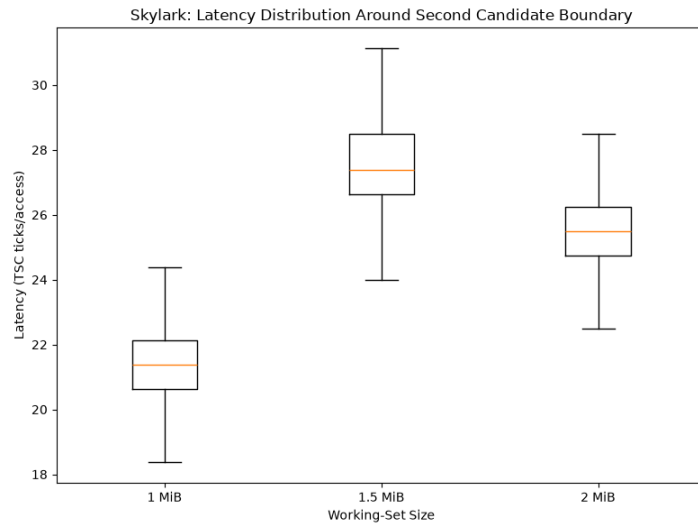


Figure 59: Skylark latency distribution around the second candidate cache boundary.

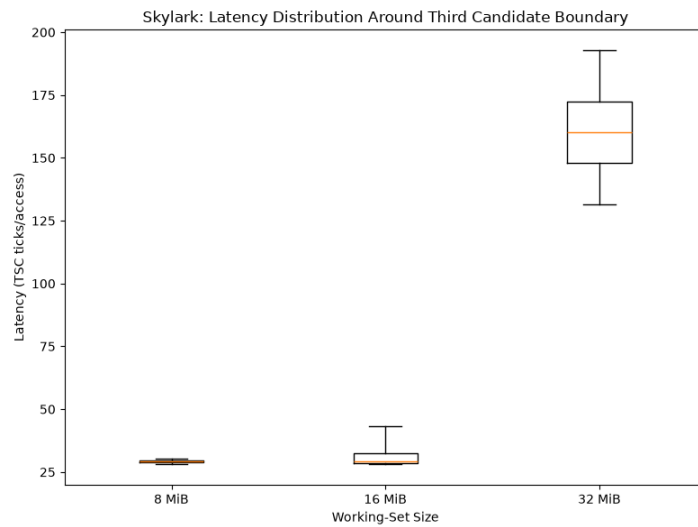


Figure 60: Skylark latency distribution around the third candidate cache boundary.

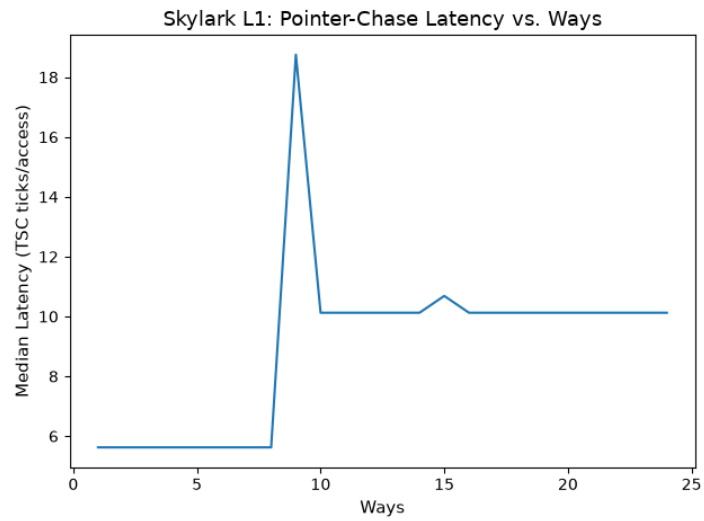


Figure 61: Skylark L1 pointer-chase latency versus number of ways.

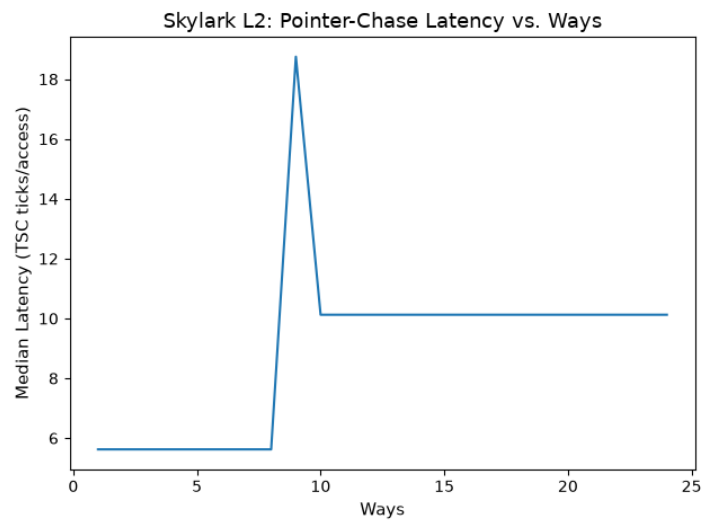


Figure 62: Skylark L2 pointer-chase latency versus number of ways.

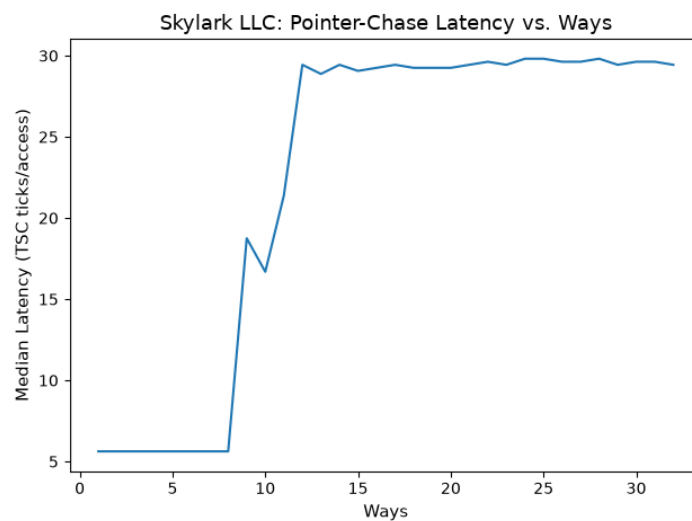


Figure 63: Skylark LLC pointer-chase latency versus number of ways.

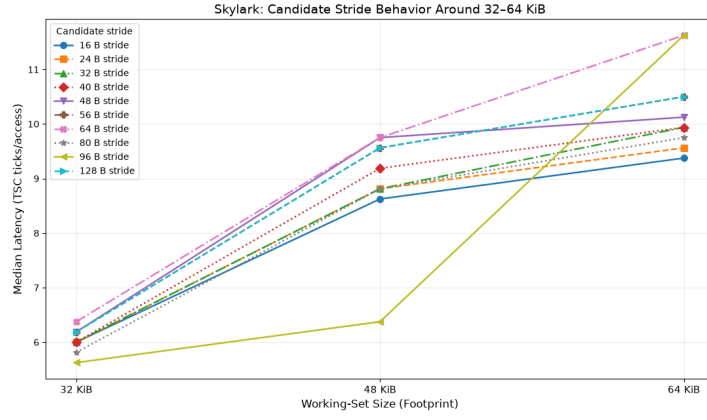


Figure 64: Skylark candidate stride behavior around the 32-64 KiB working-set range.

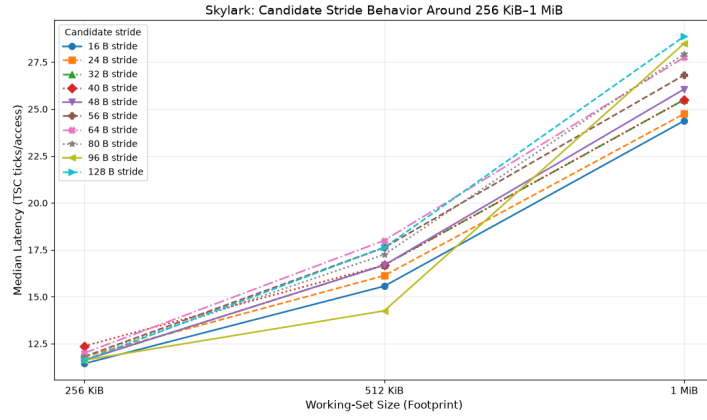


Figure 65: Skylark candidate stride behavior around the 256 KiB-1 MiB working-set range.

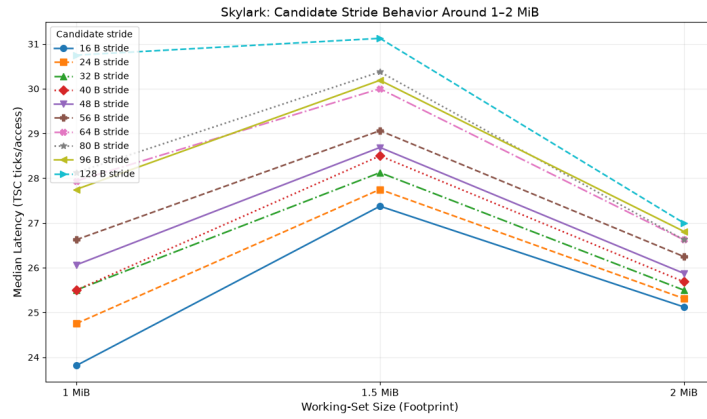


Figure 66: Skylark candidate stride behavior around the 1-2 MiB working-set range.

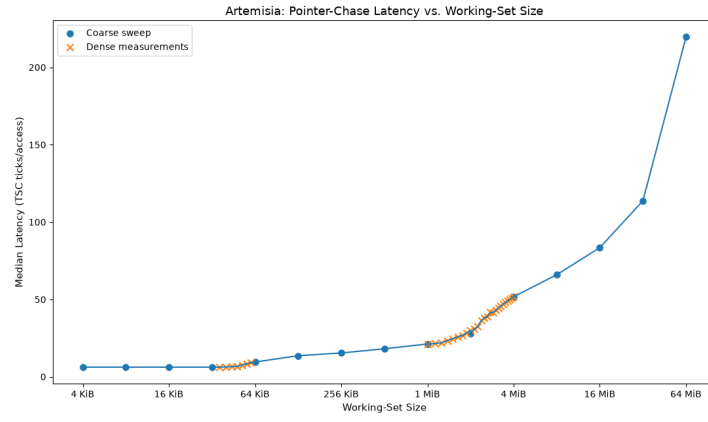


Figure 67: Artemisia coarse and dense working-set measurements.

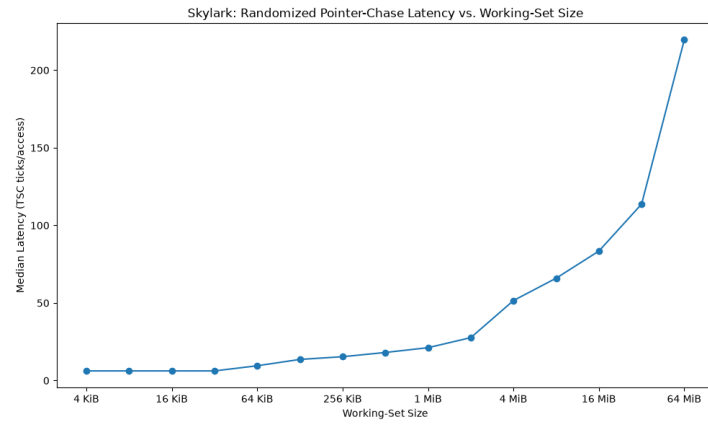


Figure 68: Artemisia coarse randomized capacity sweep.

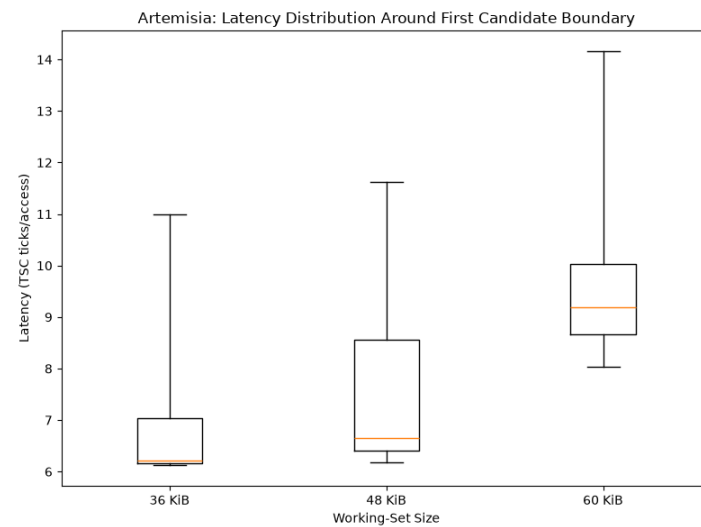


Figure 69: Artemisia latency distribution around the first candidate cache boundary.

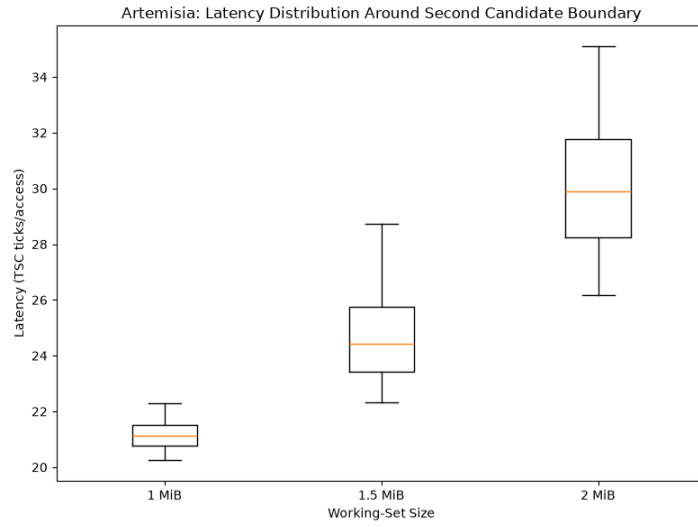


Figure 70: Artemisia latency distribution around the second candidate cache boundary.

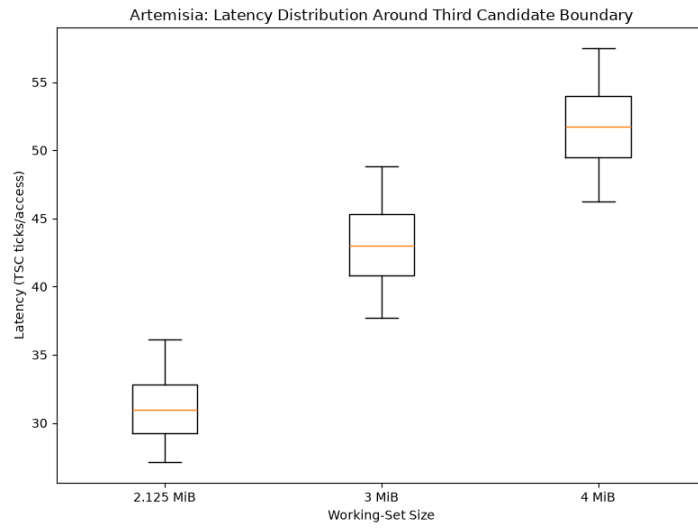


Figure 71: Artemisia latency distribution around the third candidate cache boundary.

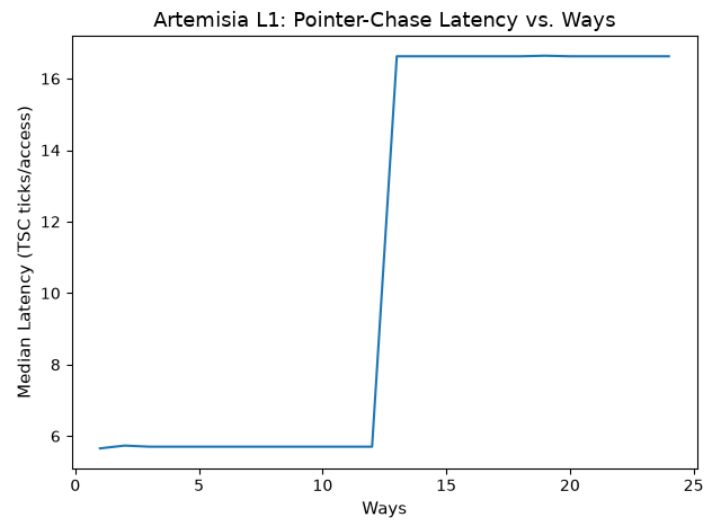


Figure 72: Artemisia L1 pointer-chase latency versus number of ways.

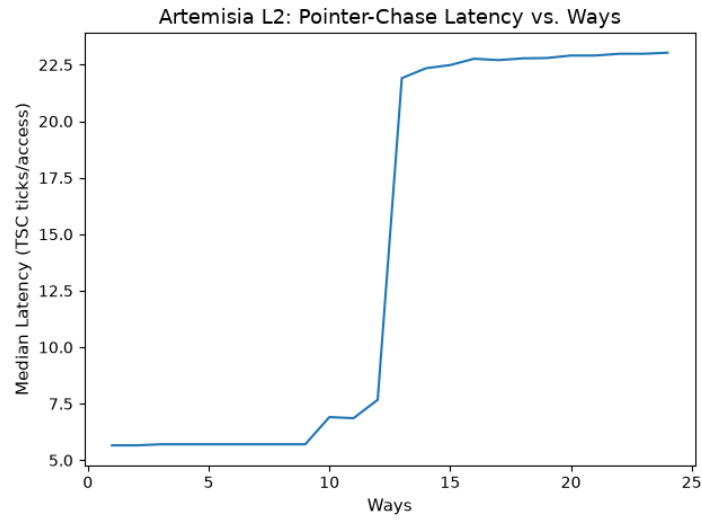


Figure 73: Artemisia L2 pointer-chase latency versus number of ways.

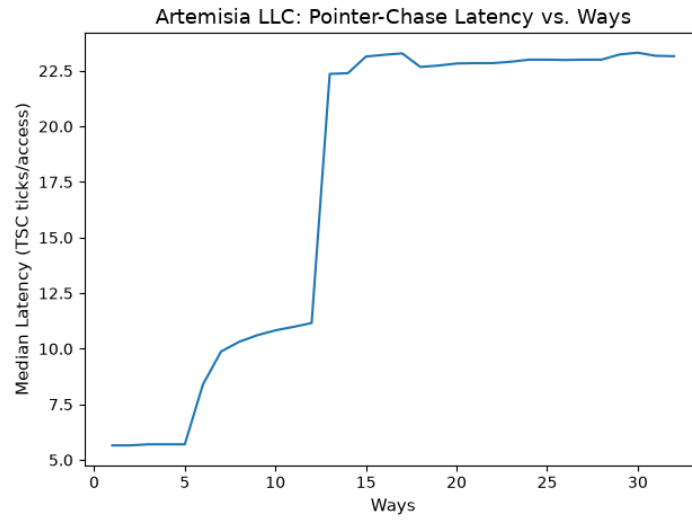


Figure 74: Artemisia LLC pointer-chase latency versus number of ways.

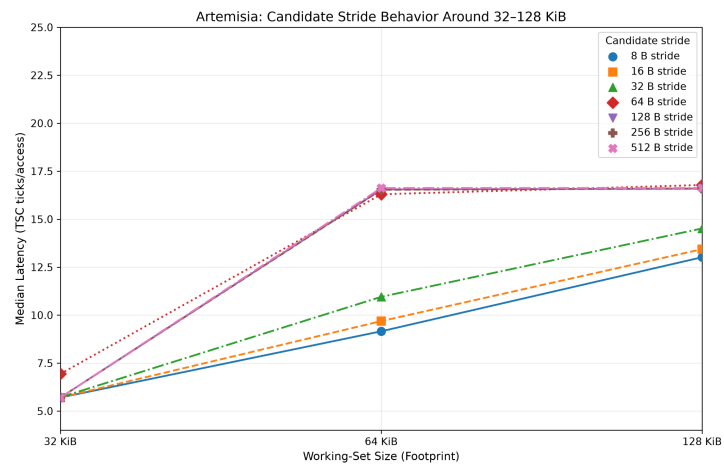


Figure 75: Artemisia candidate stride behavior around the 32–128 KiB working-set range.

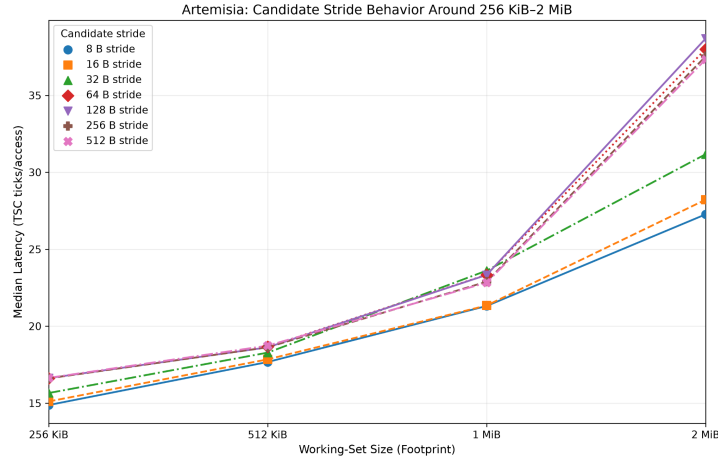


Figure 76: Artemisia candidate stride behavior around the 256 KiB–2 MiB working-set range.

## 2.6 Elevation

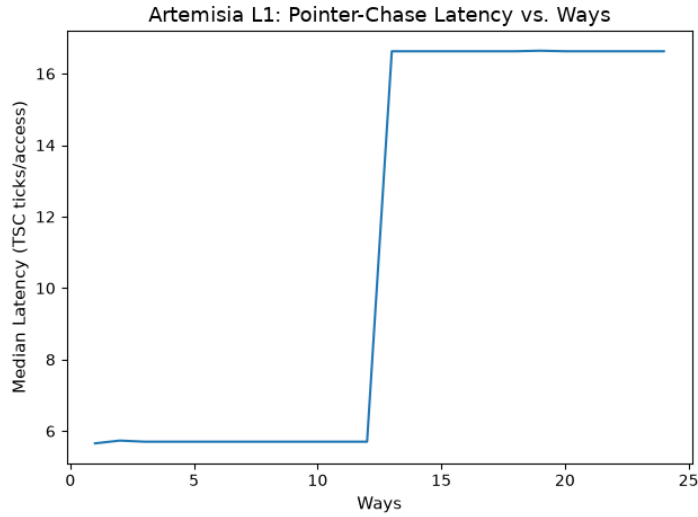


Figure 77: Artemisia candidate stride behavior around the 1–4 MiB working-set range.

## 3 Phase II

After completing our cache analysis using timing data, we compared our results to reference values. We found that our cache size analysis was mostly correct, and our line size analysis was always correct. However our associativity results were not as accurate. We predicted 4- or 8-way associativity from our timing data, but the reference values showed that some processors had an associativity of 12 or more.

### 3.1 Sunbird

| Level | Measured size | Measured ways | Derived sets | Line | Measured latency | Sharing scope | Reference value      | Agreement |
|-------|---------------|---------------|--------------|------|------------------|---------------|----------------------|-----------|
| L1D   | 32 KiB        | 8             | 64           | 64 B |                  |               | 32 KiB, 8-way, 64 B  | yes       |
| L2    | 256 KiB       | 4             | 1024         | 64 B |                  |               | 256 KiB, 4-way, 64 B | yes       |
| LLC   | 16 MiB        | 8             | 32768        | 64 B |                  | Shared        | 16 MiB, 8-way, 64 B  | yes       |

### 3.2 Elevation

| Level | Measured size | Measured ways | Derived sets | Line | Measured latency | Sharing scope | Reference value        | Agreement |
|-------|---------------|---------------|--------------|------|------------------|---------------|------------------------|-----------|
| L1D   | 64 KiB        | 8             | 128          | 64 B |                  |               | 32 KiB, 8-way, 64 B    |           |
| L2    | 2 MiB         | 8             | 4096         | 64 B |                  |               | L2 MiB, 16-way, 64 B   |           |
| LLC   | 16 MiB        | 8             | 32768        | 64 B |                  |               | 4-30 MiB, 10-way, 64 B |           |

### 3.3 Skylark

| Level | Measured size | Measured ways | Derived sets | Line | Measured latency | Sharing scope | Reference value                | Agreement |
|-------|---------------|---------------|--------------|------|------------------|---------------|--------------------------------|-----------|
| L1D   | 64 KiB        | 8             | 128          | 64 B |                  |               | 32 KiB, 8-way, 64 B            | no        |
| L2    | 1.5 MiB       | 8             | 3072         | 64 B |                  |               | 512 kB, 8-way, 1024 sets, 64 B | no        |
| LLC   | 16 MiB        | 8             | 32768        | 64 B |                  |               | 4-32 MB, 16-24 way, 64 B       | yes       |

### 3.4 Artemisia

| Level | Measured size | Measured ways | Derived sets | Line | Measured latency | Sharing scope | Reference value | Agreement |
|-------|---------------|---------------|--------------|------|------------------|---------------|-----------------|-----------|
| L1D   | 48 KiB        | 12            | 64           | 64 B |                  |               | 32 KiB          | no        |
| L2    | 2 MiB         | 12            | 2048         | 64 B |                  |               | 64 KiB          | no        |
| LLC   | 3 MiB         | 12            | 4096         | 64 B |                  |               | 4 MiB           | no        |

### 3.5 Charnwood

| Level | Measured size | Measured ways | Derived sets | Line | Measured latency | Sharing scope | Reference value | Agreement |
|-------|---------------|---------------|--------------|------|------------------|---------------|-----------------|-----------|
| L1D   | 32 KiB        | 8             | 64           | 64 B |                  |               | 32kB            | yes       |
| L2    | 256 KiB       | 4             | 1024         | 64 B |                  |               | 2048 kB         | no        |
| LLC   | 8 MiB         | 16            | 8192         | 64 B |                  |               | 4 - 30 MB       | yes       |

### 3.6 Crux

| Level | Measured size | Measured ways | Derived sets | Line | Measured latency | Sharing scope | Reference value | Agreement |
|-------|---------------|---------------|--------------|------|------------------|---------------|-----------------|-----------|
| L1D   | 32 KiB        | 8             | 64           | 64 B |                  |               | 32 kB           | yes       |
| L2    | 256 KiB       | 8             | 512          | 64 B |                  |               | 256 kB          | yes       |
| LLC   | 4 MiB         | 4             | 16384        | 64 B |                  |               | 12 MB           | no        |

### 3.7 Ookay

| Level | Measured size | Measured ways | Derived sets | Line | Measured latency | Sharing scope | Reference value | Agreement |
|-------|---------------|---------------|--------------|------|------------------|---------------|-----------------|-----------|
| L1D   | 32 KiB        | 8             | 64           | 64 B |                  |               | 32 KiB          | yes       |
| L2    | 256 KiB       | 4             | 1024         | 64 B |                  |               | 256 KiB         | yes       |
| LLC   | 4 MiB         | 4             | 16384        | 64 B |                  |               | 8 MiB           | no        |

### 3.8 Upgrade

| Level | Measured size | Measured ways | Derived sets | Line | Measured latency | Sharing scope | Reference value | Agreement |
|-------|---------------|---------------|--------------|------|------------------|---------------|-----------------|-----------|
| L1D   | 32 KiB        | 8             | 64           | 64 B |                  |               | 32 KiB          | yes       |
| L2    | 128 KiB       | 4             | 512          | 64 B |                  |               | 256 KiB         | no        |
| LLC   | 4 MiB         | 4             | 16384        | 64 B |                  |               | 12 MiB          | no        |

Table 1: Chronological summary of the timing-derived cache parameters measured during Phase I. Values shown for cache size, associativity, and line size are timing-inferred results.

| Year | Vendor / ISA   | Microarchitecture | Machine / CPU               | Process Node | L1D    |      |      | L2      |      |      | LLC    |      |      |
|------|----------------|-------------------|-----------------------------|--------------|--------|------|------|---------|------|------|--------|------|------|
|      |                |                   |                             |              | Size   | Ways | Line | Size    | Ways | Line | Size   | Ways | Line |
| 2014 | Intel / x86-64 | Haswell           | Sunbird / Xeon E5-2680 v3   | 22 nm        | 32 KiB | 8    | 64 B | 256 KiB | 4    | 64 B | 16 MiB | 8    | 64 B |
| 2015 | Intel / x86-64 | Skylake           | Charnwood / Core i7-6700    | 14 nm        | 32 KiB | 8    | 64 B | 256 KiB | 4    | 64 B | 8 MiB  | 16   | 64 B |
| 2017 | Intel / x86-64 | Kaby Lake         | Ookay / Core i7-7700        | 14 nm        | 32 KiB | 8    | 64 B | 256 KiB | 4    | 64 B | 4 MiB  | 4    | 64 B |
| 2017 | Intel / x86-64 | Coffee Lake       | Upgrade / Core i7-8700      | 14 nm        | 32 KiB | 8    | 64 B | 128 KiB | 4    | 64 B | 4 MiB  | 4    | 64 B |
| 2019 | Intel / x86-64 | Coffee Lake       | Crux / Core i7-9700         | 14 nm        | 32 KiB | 8    | 64 B | 256 KiB | 8    | 64 B | 4 MiB  | 4    | 64 B |
| 2019 | AMD / x86-64   | Zen 2 (Rome)      | Skylark / EPYC 7532         | 7 nm         | 64 KiB | 8    | 64 B | 1.5 MiB | 8    | 64 B | 16 MiB | 8    | 64 B |
| 2021 | Intel / x86-64 | Alder Lake        | Elevation / Core i5-1235U   | Intel 7      | 64 KiB | 8    | 64 B | 2 MiB   | 8    | 64 B | 16 MiB | 8    | 64 B |
| 2023 | Intel / x86-64 | Raptor Cove       | Artemisia / Xeon Gold 5420+ | Intel 7      | 48 KiB | 12   | 64 B | 2 MiB   | 12   | 64 B | 3 MiB  | 12   | 64 B |

## 4 Conclusion

In this project, we investigated the cache hierarchies of several processor generations using timing-based reverse-engineering techniques. Phase I demonstrated that important properties of a processor’s memory hierarchy can be inferred experimentally without initially relying on published processor specifications. By controlling the working-set size, access pattern, stride, and number of conflicting addresses, we were able to observe changes in memory-access latency associated with different levels of the cache hierarchy.

Our capacity experiments used randomized pointer chasing over progressively larger working sets. Distinct changes in access latency provided candidate boundaries between the L1 data cache, L2 cache, last-level cache (LLC), and main memory. Dense measurements around these candidate boundaries were then used to refine the initial estimates. Although the transitions were not always perfectly sharp, the experiments demonstrated that changes in working-set size can reveal the approximate capacities and latency regions of otherwise hidden cache structures.

We also compared randomized and sequential access patterns. Sequential accesses frequently benefited from hardware mechanisms such as spatial locality and prefetching, which could reduce the apparent cost of accessing larger working sets. Randomized pointer chasing reduced these effects and therefore provided a more useful access pattern for identifying cache capacity transitions. This comparison also demonstrated why the design of a microbenchmark is important when attempting to infer architectural properties from timing alone.

Cache-line size was investigated by varying the distance between dependent memory accesses and observing where additional memory traffic began to affect latency. Across the machines tested, our measurements consistently indicated a 64-byte cache-line size. Associativity experiments were more difficult. By constructing accesses intended to map multiple addresses to the same cache set, we attempted to identify the number of ways available before conflict misses caused a latency increase. These experiments produced useful estimates, but associativity was generally more difficult to infer reliably from timing than cache capacity or line size.

The results also illustrated an important limitation of timing-only reverse engineering. Modern processors contain prefetchers, replacement policies, shared cache structures, frequency-management mechanisms, operating-system activity, and other microarchitectural features that can influence observed latency. Consequently, a measured transition does not always correspond to a perfectly isolated cache event. Repeated measurements, randomized access patterns, processor affinity, and carefully selected working sets were therefore important for separating architectural behavior from experimental noise.

Due to the available project time, we were unable to fully complete the planned Phase II PMU-based validation and Phase III analysis. These phases would have provided additional evidence for determining whether the transitions observed during Phase I actually corresponded to specific L1, L2, LLC, TLB, or main-memory events. In particular, hardware performance counters could be used to measure level-specific cache hits and misses and compare those events directly with the timing transitions observed in our microbenchmarks. Such measurements would also help explain cases where our timing-derived associativity or cache-capacity estimates disagreed with reference values. We were able to run PMU metrics on some machines, and our raw data is included in our GitHub repository.

With additional time, this methodology could be extended beyond identifying cache capacity, associativity, line size, and latency. Additional carefully designed microbenchmarks and PMU experiments could investigate cache inclusion or exclusion behavior, replacement policies, prefetching behavior, TLB organization, memory-level parallelism, cache sharing between cores, and other characteristics of the memory hierarchy. Similar experimental techniques could also be applied to other components of the processor, allowing progressively more of the underlying microarchitecture to be reconstructed from externally observable behavior.

Overall, Phase I showed that reverse engineering provides a practical way to learn about processor architecture experimentally. Rather than treating the processor as a collection of specifications, the experiments allowed us to form hypotheses about hidden hardware structures, design measurements to test those hypotheses, and compare the resulting behavior across multiple processor generations. Although the complete planned analysis was not finished, the work completed in Phase I established a foundation from which a more comprehensive reverse engineering of the tested architectures could be performed.

## Appendix A: Team Members and GitHub Accounts

| Member Name    | Unity ID          | GitHub Username |
|----------------|-------------------|-----------------|
| Ian Ongwacho   | ibongwac@ncsu.edu | Ongwacho-I      |
| Abbott Sherwin | ajsherwi@ncsu.edu | Cohobast        |

## Appendix B: Result / Data-Point Attribution

## 5 References

- 1: <https://www.phoronix.com/review/intel-5th-gen-xeon-emeraldrapids/>
- 2: <https://www.intel.com/content/www/us/en/products/sku/88196/intel-core-i76700-processor-8m-cache-up-to-4-00-ghz/specifications.html>
- 3: <https://www.intel.com/content/www/us/en/products/sku/97128/intel-core-i77700-processor-8m-cache-up-to-4-20-ghz/specifications.html>
- 4: <https://www.intel.com/content/www/us/en/products/sku/126686/intel-core-i78700-processor-12m-cache-up-to-4-60-ghz/specifications.html>
- 5: <https://www.tomshardware.com/news/amd-epyc-7662-epyc-7532>
- 6: <https://www.techspot.com/news/95415-ampere-altra-80-core-server-processor-gets-delidded.html>
- 7: <https://uops.info/cache.html#CFL>
- 8: <https://edc.intel.com/content/www/vn/vi/de sign/products/platforms/details/raptor-lake-s/13th-generation-core-processors-datasheet-volume-1-of-2/014/ia-cores-level-1-and-level-2-caches/>

Table 2: Result/data-point attribution for reported Phase I measurements.

| Result / Data ID | Result, figure, table, machine, or data series                                                                                                                                                                             | Member 1 contribution                                                                               | Member 2 contribution                                                                               | Raw-data / script / commit reference                                       |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| R1               | Sunbird – Phase I cache characterization: capacity sweeps, dense boundary measurements, stride/line-size experiments, associativity sweeps, latency distributions, and randomized/sequential pointer-chase measurements.   | Collected measurements; processed and analyzed results; generated plots; inferred cache properties. | Reviewed results and analysis.                                                                      | Sunbird raw data, analysis scripts, and plots in team GitHub repository.   |
| R2               | Elevation – Phase I cache characterization: capacity sweeps, dense boundary measurements, stride/line-size experiments, associativity sweeps, latency distributions, and randomized/sequential pointer-chase measurements. | Collected measurements; processed and analyzed results; generated plots; inferred cache properties. | Reviewed results and analysis.                                                                      | Elevation raw data, analysis scripts, and plots in team GitHub repository. |
| R3               | Skylark – Phase I cache characterization: capacity sweeps, dense boundary measurements, stride/line-size experiments, associativity sweeps, latency distributions, and randomized/sequential pointer-chase measurements.   | Collected measurements; processed and analyzed results; generated plots; inferred cache properties. | Reviewed results and analysis.                                                                      | Skylark raw data, analysis scripts, and plots in team GitHub repository.   |
| R4               | Artemisia – Phase I cache characterization: capacity sweeps, dense boundary measurements, stride/line-size experiments, associativity sweeps, latency distributions, and randomized/sequential pointer-chase measurements. | Collected measurements; processed and analyzed results; generated plots; inferred cache properties. | Reviewed results and analysis.                                                                      | Artemisia raw data, analysis scripts, and plots in team GitHub repository. |
| R5               | Charnwood – Phase I cache characterization: capacity sweeps, dense boundary measurements, stride/line-size experiments, associativity sweeps, latency distributions, and randomized/sequential pointer-chase measurements. | Reviewed results and analysis.                                                                      | Collected measurements; processed and analyzed results; generated plots; inferred cache properties. | Charnwood raw data, analysis scripts, and plots in team GitHub repository. |
| R6               | Crux – Phase I cache characterization: capacity sweeps, dense boundary measurements, stride/line-size experiments, associativity sweeps, latency distributions, and randomized/sequential pointer-chase measurements.      | Reviewed results and analysis.                                                                      | Collected measurements; processed and analyzed results; generated plots; inferred cache properties. | Crux raw data, analysis scripts, and plots in team GitHub repository.      |
| R7               | Ookay – Phase I cache characterization: capacity sweeps, dense boundary measurements, stride/line-size experiments, associativity sweeps, latency distributions, and randomized/sequential pointer-chase measurements.     | Reviewed results and analysis.                                                                      | Collected measurements; processed and analyzed results; generated plots; inferred cache properties. | Ookay raw data, analysis scripts, and plots in team GitHub repository.     |
| R8               | Upgrade – Phase I cache characterization: capacity sweeps, dense boundary measurements, stride/line-size experiments, associativity sweeps, latency distributions, and randomized/sequential pointer-chase measurements.   | Reviewed results and analysis.                                                                      | Collected measurements; processed and analyzed results; generated plots; inferred cache properties. | Upgrade raw data, analysis scripts, and plots in team GitHub repository.   |
| R9               | Upgrade – Phase 2 PMU analysis: capacity, line size, and associativity for Charnwood, Crux, Ookay, and Upgrade                                                                                                             | Created code for PMU-based microbenchmark                                                           | Collected measurements.                                                                             | PMU results raw data in team GitHub repository.                            |

9: <https://www.agner.org/optimize/microarchitecture.pdf>

AI disclosure: `cache_capacity_x86.c`, `cache_line_assoc_x86.c`, `pmu_verify.c`, and the Python scripts for generating the graphs were created with AI assistance.